

DOCUMENTAÇÃO TÉCNICA

GSF Clubes

Plataforma de Gestão

Sistema multi-clube para gestão de Clubes de Desbravadores e Aventureiros — arquitetura, modelo de dados, segurança, perfis de acesso e módulos funcionais.

Documento

Especificação Técnica Completa

Versão do app

1.0.10 · versionCode 15

Plataformas

Web (PWA) · Android

Data

23 de agosto de 2026

§ Sumário

Estrutura completa do documento.

1. Visão Geral do Produto

- 1.1 Propósito e escopo
 - 1.2 Domínio de negócio
 - 1.3 Atores do sistema
 - 1.4 Capacidades por módulo
-

2. Arquitetura da Solução

- 2.1 Diagrama de contexto
 - 2.2 Diagrama de containers
 - 2.3 Stack tecnológico
 - 2.4 Organização do código
 - 2.5 Topologia de implantação
-

3. Modelo de Dados

- 3.1 Visão geral e domínios
 - 3.2 ER — Núcleo multi-clube e identidade
 - 3.3 ER — Pontuação e ranking
 - 3.4 ER — Classes, requisitos e especialidades
 - 3.5 ER — Atividades e planos formativos
 - 3.6 ER — Documentos, LGPD e auditoria
 - 3.7 Dicionário de dados completo
 - 3.8 DDL das tabelas centrais
-

4. Segurança e Controle de Acesso

- 4.1 Defesa em profundidade
 - 4.2 Fluxo de autenticação
 - 4.3 Modelo de contexto multi-clube
 - 4.4 Catálogo de perfis
 - 4.5 Matriz de permissões
 - 4.6 Matriz de perfis × telas
 - 4.7 Row Level Security no Postgres
 - 4.8 Funções de segurança (SECURITY DEFINER)
 - 4.9 Storage e políticas de arquivos
-

5. Módulos Funcionais

- 5.1 Membros e unidades
- 5.2 Pontuação
- 5.3 Pontos extras e descontos
- 5.4 Ranking
- 5.5 Classes e requisitos

- 5.6 Especialidades
 - 5.7 Investidura e validação
 - 5.8 Documentos
 - 5.9 Atividades e planos formativos
 - 5.10 Comunicação
 - 5.11 Calendário
 - 5.12 Pré-cadastro público
 - 5.13 Relatórios e importação
 - 5.14 Administração da plataforma
-

6. Sincronização Offline-First

- 6.1 Estratégia local-first
 - 6.2 Fluxo de descida (pull)
 - 6.3 Fluxo de subida (push)
 - 6.4 Reconciliação de identificadores
 - 6.5 Remoção de órfãos
 - 6.6 Resolução de conflitos
-

7. Funções, Triggers e Automações de Banco

- 7.1 Catálogo de funções
 - 7.2 Triggers e cadeias de automação
 - 7.3 RPCs consumidas pelo app
-

8. Build, Deploy e Operação

- 8.1 Ambientes
 - 8.2 Pipeline web
 - 8.3 Pipeline Android
 - 8.4 Migrações de banco
 - 8.5 Rotina operacional
-

9. Decisões Arquiteturais (ADRs)

10. Anexos

- 10.1 Índice de migrações
 - 10.2 Índice de rotas
 - 10.3 Glossário
-

1 Visão Geral do Produto

O que o sistema faz, para quem, e qual problema de negócio resolve.

1.1 Propósito e escopo

O **GSF Clubes** é uma plataforma de gestão para Clubes de Desbravadores e Aventureiros da Igreja Adventista do Sétimo Dia. O sistema substitui o controle manual em planilhas por um ambiente único, auditável e sincronizado, cobrindo o ciclo completo de vida do membro: do pré-cadastro público à investidura em classes e especialidades.

A plataforma é **multi-clube e multi-programa** por construção: um mesmo banco de dados atende vários clubes, cada um vinculado a um programa (Desbravadores, faixa 10–15 anos; ou Aventureiros, faixa 6–9 anos), com isolamento de dados garantido no nível do banco por *Row Level Security*. Um mesmo usuário pode ter papéis diferentes em clubes diferentes e alternar entre eles sem trocar de conta.

Princípio orientador

O clube opera em campo — acampamentos, marchas, atividades ao ar livre — onde a conectividade é intermitente. Toda a aplicação foi construída **offline-first**: o aparelho grava localmente e sincroniza quando houver rede, sem bloquear o usuário.

1.2 Domínio de negócio

Um clube organiza seus membros em **unidades** (pequenos grupos com nome próprio, como “Águia Dourada” ou “Amor Perfeito”), lideradas por conselheiros. Cada reunião gera **pontuação** individual (presença, pontualidade, material, uniforme, além de itens configuráveis por clube) que alimenta um **ranking** individual e por unidade.

Paralelamente, cada membro percorre uma trilha formativa composta por **classes** (Amigo → Companheiro → Pesquisador → Pioneiro → Excursionista → Guia, mais as avançadas e as classes de liderança) e por **especialidades** (mais de mil itens de catálogo, agrupados por categoria). O sistema modela os requisitos individuais de cada classe, o progresso de cada membro em cada requisito, e a validação final pela diretoria — a **investidura**.

1.3 Atores do sistema

Ator	Escopo	Responsabilidade principal no sistema
Admin TI	Plataforma	Opera a plataforma inteira: cria clubes, gerencia catálogos globais (classes, especialidades, requisitos), acessa auditoria de todos os clubes.
Admin do Clube	Clube	Autoridade máxima dentro de um clube: acessos, membros, pontuação, unidades, aparência e relatórios.
Diretoria	Clube	Operação ampla do clube — lança pontuação, gere membros e unidades, valida classes.
Secretaria	Clube	Cadastro de membros, gestão documental e emissão de relatórios.
Tesouraria	Clube	Visão financeira e relatórios de pagamento.
Conselheiro(a)	Unidade	Acompanha os membros da sua unidade, lança pontuação e valida requisitos de classe.
Instrutor(a)	Unidade	Mesmo acompanhamento formativo do conselheiro, porém sem poder editar fichas de terceiros.
Capelão	Clube	Acompanhamento espiritual: cria atividades devocionais e envia mensagens.
Regional / Distrital / Pastor	Supra-clube	Acompanhamento agregado de vários clubes; o Regional valida classes e especialidades.
Pais / Responsável	Responsável	Acompanha exclusivamente os filhos vinculados; envia documentos e responde atividades conforme permissões concedidas.
Desbravador / Aventureiro	Próprio	Consulta a própria ficha, pontuação, extrato, classes e atividades.

1.4 Capacidades por módulo

Módulo	Capacidades entregues
Membros	Cadastro completo (dados pessoais, responsáveis, tamanhos de uniforme, foto), cargos por programa, cargo adicional, inativação com histórico preservado, vínculo com login.
Unidades	CRUD de unidades com cor identitária, movimentação de membros, extrato de pontuação por unidade, ranking coletivo.
Pontuação	Lançamento por data com itens padrão e itens personalizados por clube; pontos extras nominais; descontos; extrato detalhado por membro.
Ranking	Classificação individual (com recortes por grupo) e por unidade; ranking entre clubes com níveis e estrelas.
Classes	Catálogo de requisitos por classe/seção/código, progresso por requisito, respostas em texto e upload, requisitos compartilhados, classes agrupadas por faixa etária, trilha de liderança.
Especialidades	Catálogo com categoria/subcategoria e insígnia; marcação manual auditada; marcação automática por conclusão de atividade.
Atividades	Criação com destino segmentado, prazo com timezone, anexos, fluxo de entrega → correção → aprovação, mensagens por atividade, reabertura controlada.
Documentos	Checklist configurável por clube, múltiplos anexos por campo, permissões diferenciadas para responsáveis.
Comunicação	Mural do clube com controle de leitura e ocultação; notificações push; integração com fila de WhatsApp.
Governança	Auditoria de eventos, aceite de termo LGPD versionado, MFA obrigatório para administradores, pré-cadastro público com consentimento.

2 Arquitetura da Solução

Como as partes se encaixam: contexto, containers, tecnologias e implantação.

2.1 Diagrama de contexto

Visão de mais alto nível: quem usa o sistema e com quais serviços externos ele conversa.

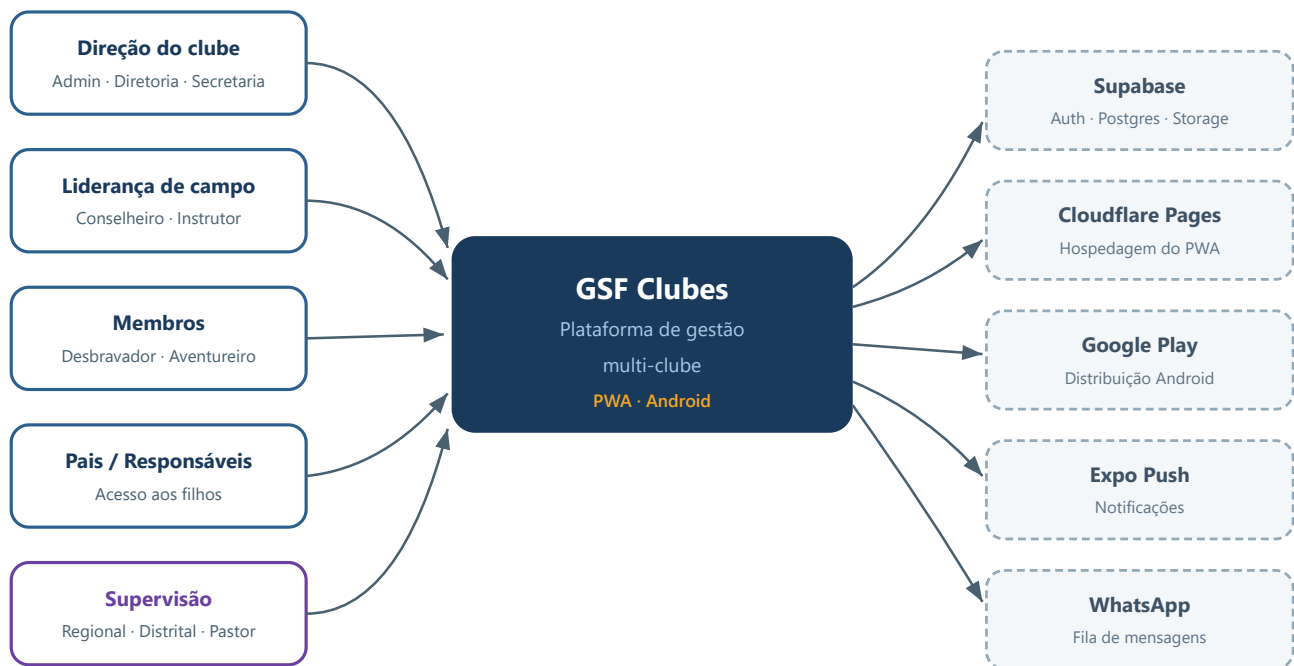


Figura 2.1 — Diagrama de contexto: atores humanos à esquerda, serviços externos à direita.

2.2 Diagrama de containers

Detalhando o interior do sistema: as unidades executáveis e os armazenamentos, com os protocolos entre eles. Note que o **mesmo código-fonte** gera tanto o PWA quanto o app Android — a diferença está apenas na camada de persistência local.

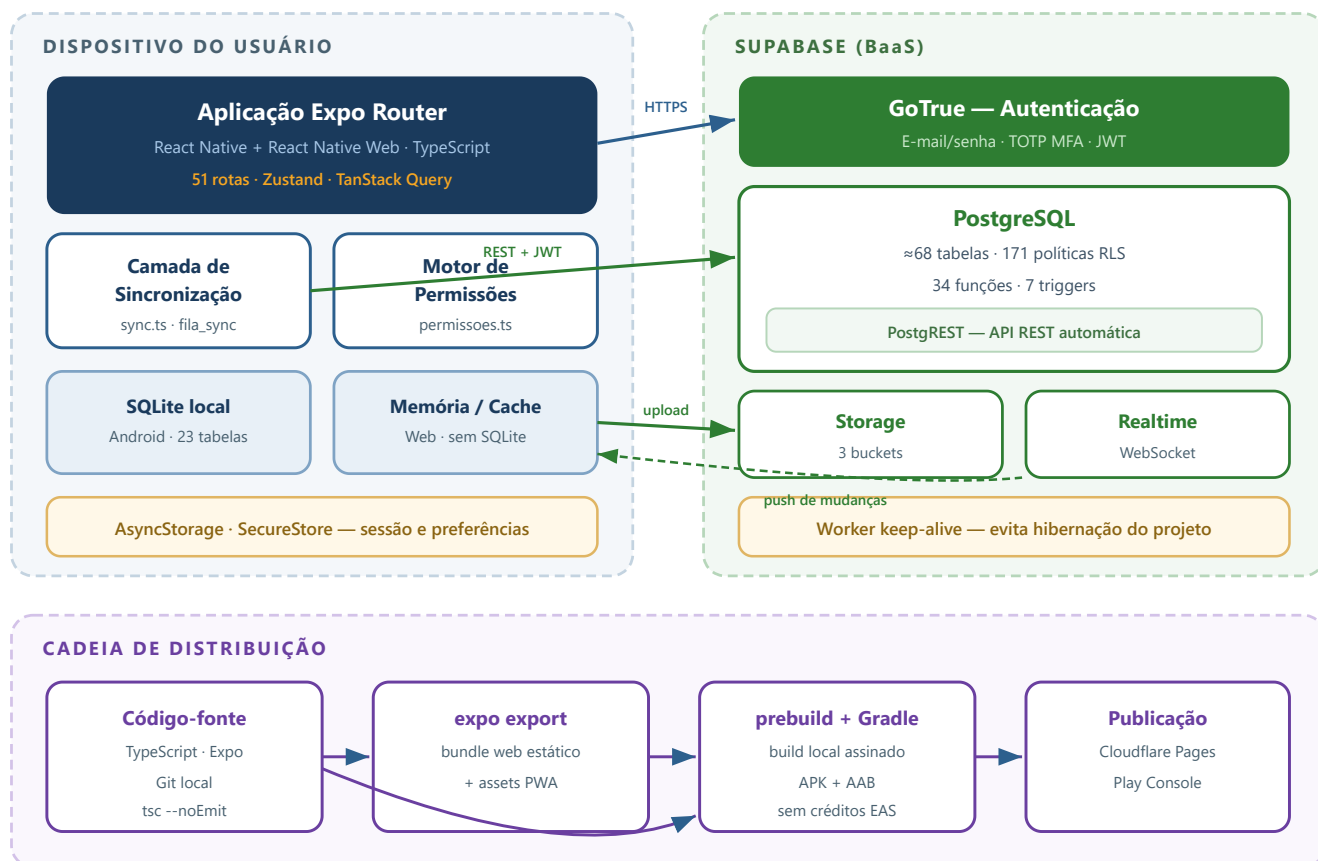


Figura 2.2 — Diagrama de containers: cliente, backend gerenciado e cadeia de distribuição.

2.3 Stack tecnológico

Camada	Tecnologia	Papel no sistema
Cliente	expo + expo-router	Runtime e roteamento por sistema de arquivos; uma base de código para Web e Android.
	react-native / react-native-web	Camada de UI unificada — os mesmos componentes renderizam nativo e DOM.
	zustand	Estado global (auth, contexto, membros, pontuação, sincronia) sem boilerplate.
	@tanstack/react-query	Cache e revalidação de dados remotos.
	expo-sqlite	Banco local no Android — base do modo offline.
	react-native-gifted-charts, xlsx, expo-print	Gráficos de ranking, importação de planilhas e geração de PDF.
Backend	Supabase Auth (GoTrue)	Autenticação por e-mail/senha e segundo fator TOTP.
	PostgreSQL + PostgREST	Banco relacional e API REST derivada do esquema; regra de negócio crítica reside aqui.
	Supabase Storage	Fotos de membros, anexos de documentos e de atividades.
	Supabase Realtime	Atualização ao vivo de telas abertas (extrato, pontuação, mural).
Infra	Cloudflare Pages	Hospedagem estática do PWA com deploy por CLI (wrangler).
	Cloudflare Workers	Keep-alive periódico impedindo a hibernação do projeto Supabase gratuito.
	Gradle + Android SDK	Build local assinado do APK/AAB, sem consumir créditos de nuvem.

2.4 Organização do código

```
fonseca-app/
├─ app/
│   └─ (tabs)/
│       └─ admin/
│           └─ auth/
│               └─ classes/
│                   └─ especialidades/
│                       └─ membro/[id].tsx
│                           └─ extrato/[dbv_id]
│                               └─ pre-cadastro/
│                                   └─ convite/[token]
├─ src/
│   └─ lib/
│       └─ stores/
│           └─ components/
│               └─ types/
├─ supabase/migrations/
├─ scripts/
├─ workers/
└─ plugins/
```

→ Rotas (expo-router, file-based routing) – 51 telas
Navegação principal: início, ranking, membros, pontuação, extras, unidades, atividades, mensagens
Painéis administrativos (17 telas)
Login, MFA, consentimento LGPD, escolha de contexto
Trilha formativa: hub, requisitos, catálogo, envio
Catálogo e conquistas
Ficha completa do membro (abas)
Extrato de pontuação individual
Formulário público (sem login)
Aceite de convite de responsável

Núcleo técnico: sync, database, supabase, permissões, classesRequisitos, especialidades, auditoria, lgpd
Estado global Zustand (auth, contexto, dbv, pontuação)
UI compartilhada
Contratos TypeScript do domínio
80 migrações numeradas e idempotentes
Automação de build, deploy e manutenção (PowerShell/Node)
Cloudflare Worker de keep-alive
Config plugin Expo – assinatura de release

Convenção importante

A camada `src/lib/` concentra as decisões técnicas sensíveis (sincronização, permissões, acesso ao banco). Telas em `app/` consomem essa camada e não falam diretamente com o Supabase quando existe um serviço equivalente — isso mantém a regra em um lugar só e evita que correções precisem ser replicadas em várias telas.

2.5 Topologia de implantação

Artefato	Destino	Observações
Bundle web	Cloudflare Pages — projeto <code>gsf-clubes</code>	Deploy por <code>wrangler pages deploy dist</code> . Alias estável em <code>master.gsf-clubes.pages.dev</code> .
App Android	Google Play Console	AAB assinado com keystore própria (SHA-1 <code>39:70:1A:...:37:0E</code>). Upload manual.
Banco e Auth	Projeto Supabase	Migrações aplicadas manualmente via SQL Editor, em ordem numérica.
Keep-alive	Cloudflare Workers	Requisição periódica que evita a pausa automática do projeto no plano gratuito.

3 Modelo de Dados

Aproximadamente 68 tabelas organizadas em seis domínios, com isolamento por clube aplicado no banco.

3.1 Visão geral e domínios

O esquema evoluiu em duas fases. A fase inicial (migrações 001 – 009) modelava um **clube único**: `desbravadores`, `unidades`, `pontuacoes`, `documentos` e `especialidades` sem qualquer noção de tenant. A migração 010_multiclube_base introduziu a fase atual, adicionando `programas`, `clubes`, `perfis_acesso` e `usuario_clubes`, e propagando a coluna `clube_id` para todas as tabelas transacionais.

Dívida técnica conhecida

A migração multi-clube foi **aditiva e não destrutiva** — a tabela legada `usuarios` e sua coluna `perfil` continuam existindo ao lado de `usuario_clubes`. O código trata isso normalizando perfis legados (`admin_total` → `admin_ti`, `admin_geral` → `admin_clube`) em `src/lib/permissoes.ts`. Toda função de segurança no banco também consulta as duas fontes.

Domínio	Tabelas	Responsabilidade
Identidade e tenancy	8	Programas, clubes, perfis de acesso, vínculos usuário↔clube, responsáveis, convites, onboarding.
Membros	6	Cadastro de membros, unidades, documentos, anexos, pré-cadastros.
Pontuação	8	Lançamentos diários, itens configuráveis, extras nominais, pontuação de unidade, ranking entre clubes.
Trilha formativa	14	Catálogo de classes e requisitos, progresso, respostas, arquivos, especialidades, investidura.
Atividades	9	Atividades avaliativas, alvos, respostas, anexos, mensagens, planos formativos.
Governança	7	Auditoria, termos e aceites LGPD, tokens de push, fila de WhatsApp, configurações visuais.
Catálogos-modelo	7	Modelos por programa: classes, cargos, documentos, especialidades, requisitos MDA, relatórios.

3.2 ER — Núcleo multi-clube e identidade

Este é o coração do modelo de tenancy. Um `programa` define a faixa etária e o catálogo base; um `clube` pertence a exatamente um programa; e `usuario_clubes` é a tabela associativa que concede a um usuário um **perfil** dentro de um clube — podendo opcionalmente amarrá-lo a uma unidade ou a um membro específico.

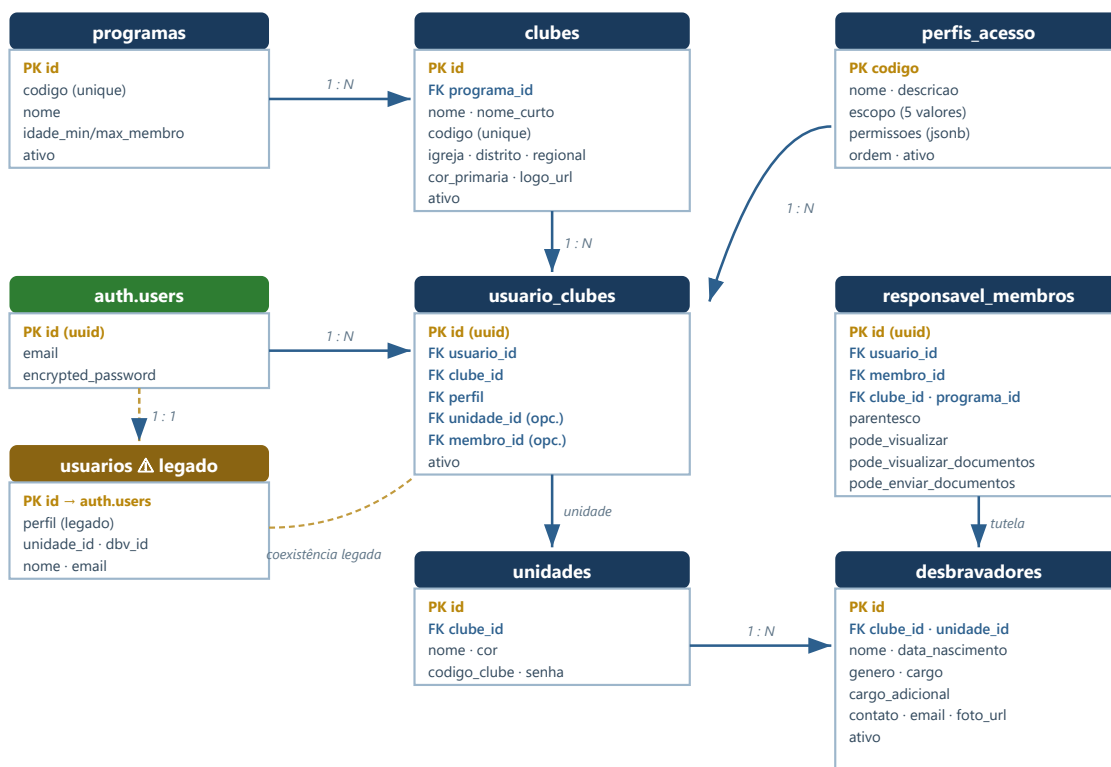


Figura 3.1 — Núcleo de identidade e tenancy. Em âmbar, a tabela legada que coexiste com o modelo novo.

Por que `usuario_clubes` é a chave do modelo

Cada linha representa **um contexto de acesso**. Um mesmo usuário pode ter várias linhas: conselheiro no Clube A, pai no Clube A e secretaria no Clube B. Ao entrar, o app carrega todos os contextos e o usuário escolhe em qual atuar. As permissões efetivas são a **união** de todos os contextos *daquele mesmo clube* — quem é conselheiro e pai simultaneamente não precisa alternar de contexto para ver os dois conjuntos de tela.

3.3 ER — Pontuação e ranking

Há três mecanismos distintos de pontuação, deliberadamente separados: os **itens padrão** (colunas fixas em `pontuacoes`), os **itens configuráveis por clube** (`pontuacoes_custom`, uma linha por item) e os **pontos extras de texto livre** (`pontuacoes_extras_itens`, uma linha por lançamento).

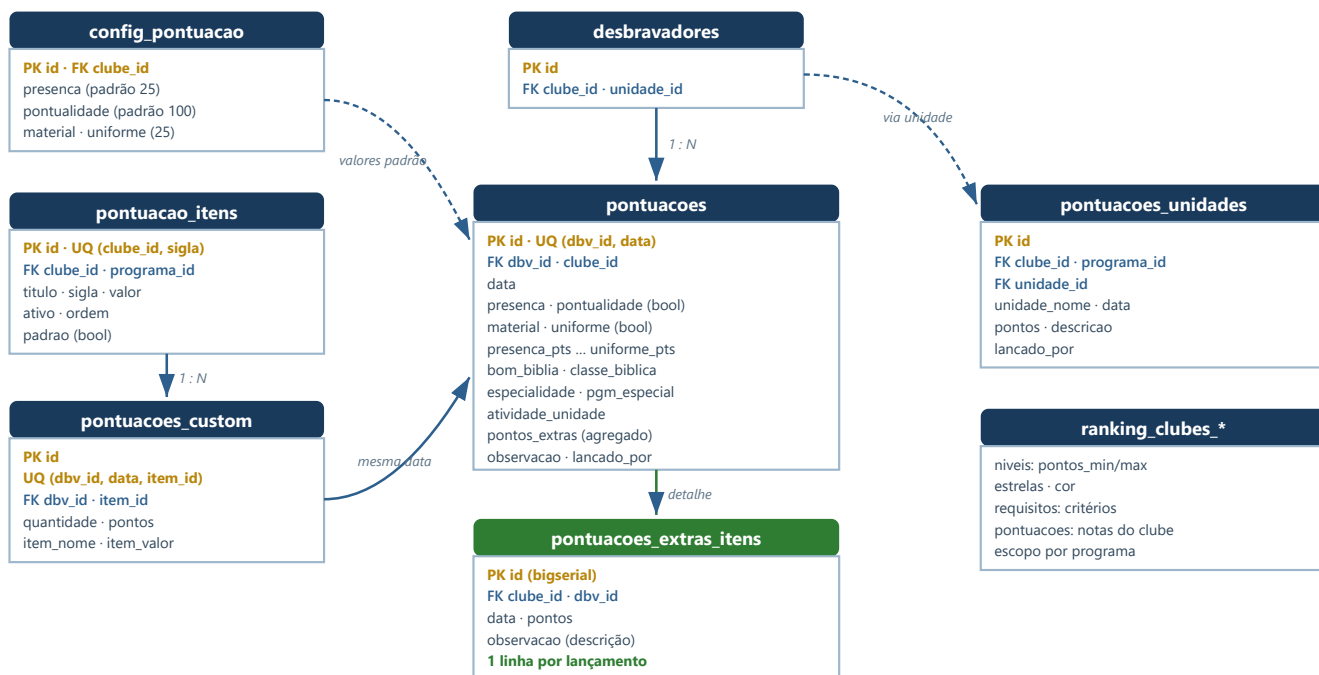


Figura 3.2 — Domínio de pontuação. Em verde, a tabela introduzida para preservar a descrição de cada ponto extra.

Decisão de design: por que `pontos_extras` permanece

A coluna agregada `pontuacoes.pontos_extras` é **mantida em dia pela aplicação** (soma/subtrai a cada lançamento, edição ou exclusão) mesmo depois da criação de `pontuacoes_extras_itens`. O motivo é minimizar o raio de impacto: cálculos de total, ranking e extrato offline já liam essa coluna diretamente. A tabela nova passou a ser a fonte da *descrição por lançamento*; a coluna continua sendo a fonte do *total*.

3.4 ER — Classes, requisitos e especialidades

É o domínio mais elaborado. O **catálogo** é global (compartilhado por todos os clubes) e descreve cada requisito de cada classe; o **progresso** é por clube e por membro. Um requisito pode ser satisfeito manualmente, por conclusão de atividade ou por posse de uma especialidade — e triggers no banco propagam essas equivalências automaticamente.

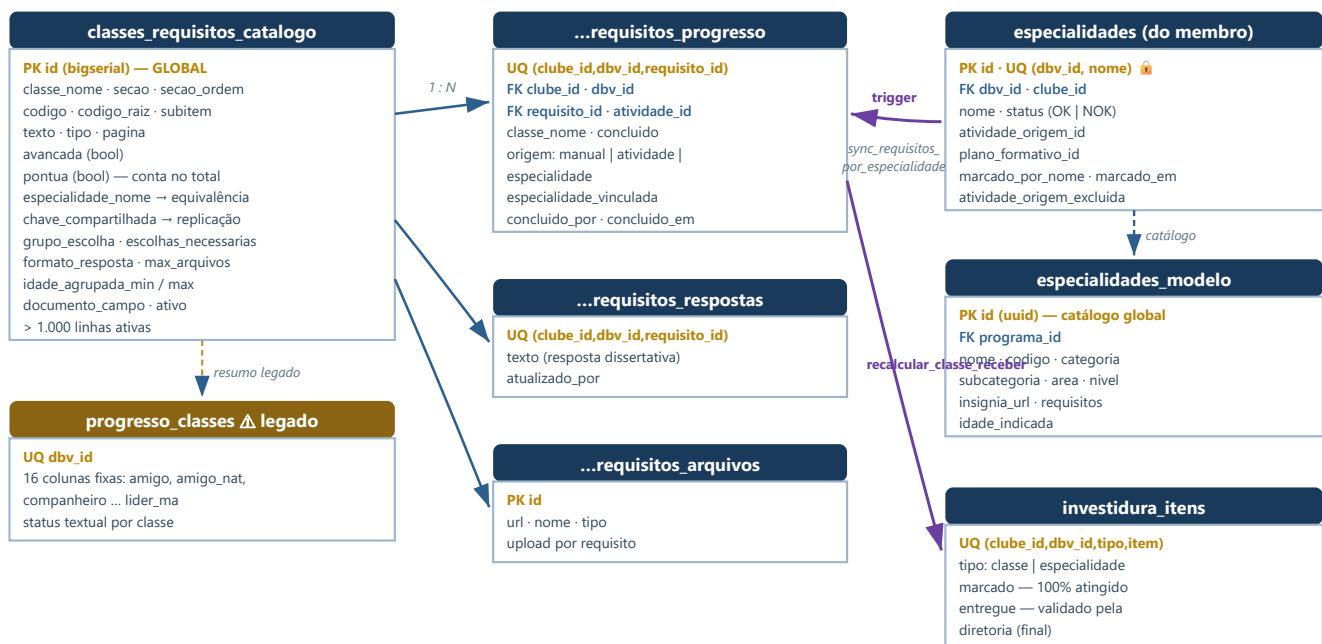


Figura 3.3 — Trilha formativa. Em roxo, as automações por trigger que propagam conclusões entre entidades.

3.5 ER — Atividades e planos formativos

Uma **atividade** é uma tarefa avaliativa direcionada a todos, a uma unidade ou a um membro. Quando vinculada a um item formativo (classe ou especialidade) por meio de um **plano formativo**, sua aprovação alimenta automaticamente o progresso de requisitos.

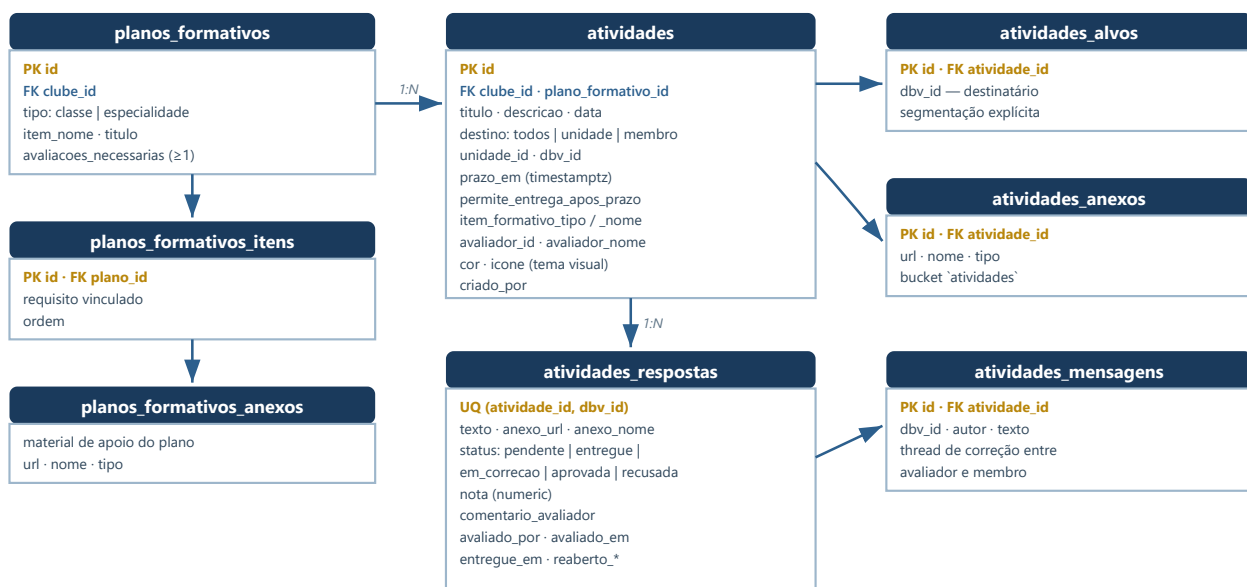


Figura 3.4 — Domínio de atividades avaliativas e sua ligação com a trilha formativa.

3.6 ER — Documentos, LGPD e auditoria

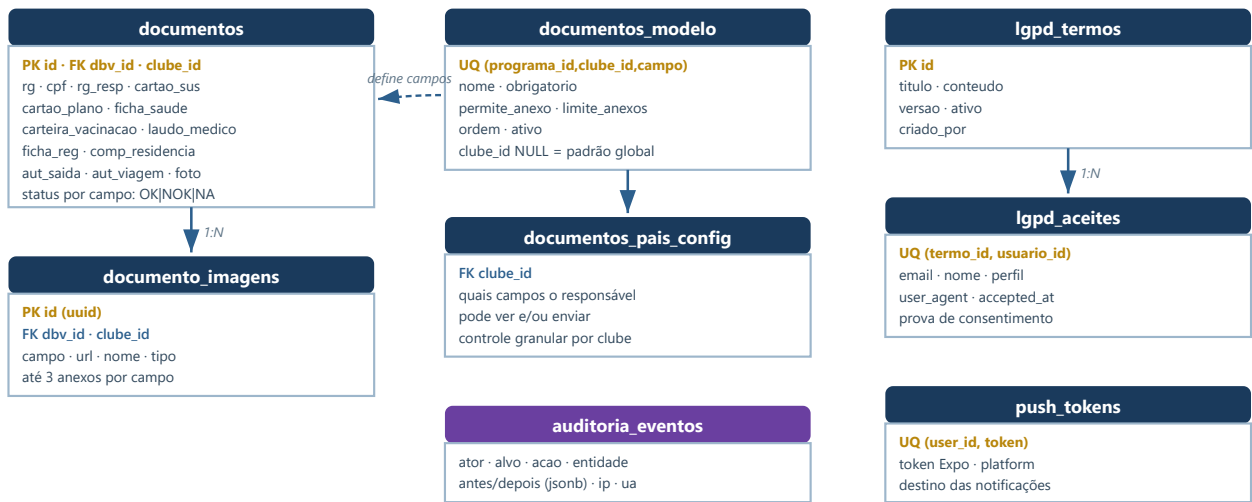


Figura 3.5 — Documentação do membro, consentimento LGPD e trilha de auditoria.

3.7 Dicionário de dados completo

Relação de todas as tabelas do esquema `public`, agrupadas por domínio.

Identidade, tenancy e acesso

Tabela	PK	Descrição
<code>programas</code>	serial	Desbravadores (10–15) e Aventureiros (6–9). Define faixas etárias e ancora todos os catálogos-modelo.
<code>clubes</code>	serial	Tenant principal. Guarda identidade visual (cores, logo) e localização (igreja, distrito, regional).
<code>perfis_acesso</code>	text	Catálogo de perfis com escopo (<code>plataforma</code> , <code>clube</code> , <code>unidade</code> , <code>proprio</code> , <code>responsavel</code>) e permissões em JSONB.
<code>usuario_clubes</code>	uuid	Tabela de contextos. Vincula usuário + clube + perfil, opcionalmente restrito a unidade ou membro.
<code>usuarios</code>	uuid	Perfil legado espelhando <code>auth.users</code> . Mantida por compatibilidade; ainda consultada pelas funções de segurança.
<code>responsavel_membros</code>	uuid	Tutela: liga um responsável a um membro com flags granulares de visualização e envio.
<code>responsavel_convites</code>	uuid	Convites por token/telefone para que um responsável crie acesso e seja vinculado.
<code>clubes_onboarding_status</code>	int	Checklist de implantação do clube (modelos clonados, unidades criadas, admin convidado).

Membros e documentação

Tabela	PK	Descrição
desbravadores	serial	Membro do clube (nome mantido por legado; abrange também aventureiros e diretoria). Inclui cargo, cargo adicional, contato, foto e flag de ativo.
unidades	serial	Subgrupo do clube, com nome e cor. Texto livre por clube.
documentos	serial	Uma linha por membro; cada campo documental guarda status OK / NOK / NA .
documento_imagens	uuid	Anexos por campo documental (limite de 3), apontando para o Storage.
documentos_modelo	serial	Define quais campos documentais existem — global por programa ou sobrescrito por clube.
documentos_pais_config	serial	Por clube: quais campos o responsável pode visualizar e enviar.
documento_tipos · documento_status	serial	Catálogos auxiliares de tipos e estados documentais.
pre_cadastro_links	uuid	Link público tokenizado, com validade e ativação, por clube.
pre_cadastros	uuid	Submissão pública. Estados: novo → em_analise → aprovado/rejeitado → convertido .
pre_cadastro_responsaveis	uuid	Múltiplos responsáveis declarados no formulário público.
importacoes_lote · _itens	uuid	Rastreo de importações em massa (planilhas) e o resultado item a item.

Pontuação e ranking

Tabela	PK	Descrição
pontuacoes	serial	Lançamento diário por membro. Única por (dbv_id, data) . Guarda itens padrão e o agregado de extras.
config_pontuacao	int	Valores padrão dos quatro itens fixos, por clube.
pontuacao_itens	serial	Itens de pontuação por clube com título, sigla e valor. Única por (clube_id, sigla) .
config_pontuacao_itens	serial	Variante de catálogo de itens usada no fluxo de pontuação personalizada.
pontuacoes_custom	serial	Uma linha por item personalizado lançado. Única por (dbv_id, data, item_id) .
pontuacoes_extras_itens	bigserial	Uma linha por lançamento de ponto extra de texto livre, preservando a descrição individual.
pontuacoes_unidades	bigserial	Pontuação atribuída à unidade como coletivo, independente dos membros.
ranking_clubes_niveis	serial	Faixas de classificação entre clubes (pontos mínimos/máximos, estrelas, cor).
ranking_clubes_requisitos · _pontuacoes	serial	Critérios avaliados e notas atribuídas a cada clube.
config_campori · parcelas_campori_config · pagamentos_campori	serial	Controle financeiro de eventos: parcelas configuradas e pagamentos por membro.

Trilha formativa

Tabela	PK	Descrição
<code>classes_requisitos_catalogo</code>	bigserial	Catálogo global de requisitos, com hierarquia (código-raiz/subitem), equivalências e regras de resposta.
<code>classes_requisitos_progresso</code>	bigserial	Conclusão por membro e requisito, com a origem registrada.
<code>classes_requisitos_respostas</code>	bigserial	Resposta dissertativa por requisito.
<code>classes_requisitos_arquivos</code>	bigserial	Uploads de comprovação por requisito.
<code>classes_modelo</code>	serial	Nomes e ordem das classes de cada programa.
<code>classes_nomes_avancadas</code>	serial	Mapeamento classe base → nome da classe avançada.
<code>progresso_classes</code>	serial	Resumo legado com 16 colunas fixas de status por classe.
<code>especialidades</code>	serial	Especialidades conquistadas pelo membro. Única por <code>(dbv_id, nome)</code> .
<code>especialidades_modelo</code>	uuid	Catálogo global por programa: categoria, subcategoria, insígnia, requisitos.
<code>mda_requisitos_modelo</code>	uuid	Requisitos oficiais importados do Manual do Desbravador para classes e especialidades.
<code>investidura_itens</code>	bigserial	Estado de validação: <code>marcado</code> (100% cumprido) e <code>entregue</code> (validado pela diretoria).
<code>classe_biblica_respostas</code>	bigserial	Respostas do módulo de classe bíblica.

Atividades, comunicação e governança

Tabela	PK	Descrição
<code>atividades</code>	uuid/bigint	Tarefa avaliativa com destino, prazo com timezone, tema visual e vínculo formativo.
<code>atividades_alvos</code>	uuid	Destinatários explícitos quando a segmentação não é “todos”.
<code>atividades_respostas</code>	uuid	Entrega do membro e o ciclo de avaliação (status, nota, comentário, reabertura).
<code>atividades_anexos</code>	uuid	Materiais anexados à atividade.
<code>atividades_mensagens</code>	uuid	Conversa entre avaliador e membro dentro da atividade.
<code>planos_formativos</code> · <code>_itens</code> · <code>_anexos</code>	bigserial	Agrupam atividades que, somadas, concluem uma classe ou especialidade.
<code>mensagens_clube</code>	uuid	Mural de comunicados do clube.
<code>mensagens_clube_lidos</code> · <code>_ocultos</code>	uuid	Controle por usuário de leitura e de ocultação de mensagens.
<code>eventos</code>	serial	Calendário do clube: data, horário, local, responsável, material.
<code>auditoria_eventos</code>	uuid	Trilha imutável: ator, alvo, ação, estado antes/depois em JSONB, IP e user-agent.
<code>lgpd_termos</code> · <code>lgpd_aceites</code>	bigserial	Termo versionado e prova de aceite por usuário.
<code>push_tokens</code>	uuid	Tokens Expo para notificação, por usuário e plataforma.
<code>whatsapp_config</code> · <code>whatsapp_fila</code>	uuid	Configuração e fila de mensagens para disparo externo.
<code>configuracoes_visuais_clube</code>	serial	Personalização de tema e paleta de atividades por clube.
<code>cargos_modelo</code> · <code>relatorios_modelo</code>	serial	Catálogos-modelo de cargos por programa e de relatórios disponíveis.
<code>arquivos_registro</code> · <code>membros</code>	—	Tabelas auxiliares de registro de arquivos e de compatibilidade de nomenclatura.

3.8 DDL das tabelas centrais

Definições reais extraídas das migrações, com as restrições que sustentam a integridade do modelo.

Tenancy

```
CREATE TABLE public.programas (  
  id SERIAL PRIMARY KEY,  
  codigo TEXT NOT NULL UNIQUE, -- 'desbravadores' | 'aventureiros'  
  nome TEXT NOT NULL,  
  idade_minima_membro INTEGER NOT NULL,  
  idade_maxima_membro INTEGER NOT NULL,  
  idade_minima_diretoria INTEGER NOT NULL DEFAULT 16,  
  ativo BOOLEAN NOT NULL DEFAULT TRUE  
);  
  
CREATE TABLE public.clubes (  
  id SERIAL PRIMARY KEY,  
  programa_id INTEGER NOT NULL REFERENCES public.programas(id),  
  nome TEXT NOT NULL,  
  codigo TEXT UNIQUE,  
  igreja TEXT, distrito TEXT, regional TEXT,  
  cidade TEXT, uf TEXT, logo_url TEXT,  
  cor_primaria TEXT DEFAULT '#1a3a5c',  
  cor_secundaria TEXT DEFAULT '#f5a623',  
  ativo BOOLEAN NOT NULL DEFAULT TRUE  
);  
  
-- Tabela de contextos: o coração do modelo de acesso  
CREATE TABLE public.usuario_clubes (  
  id UUID PRIMARY KEY DEFAULT uuid_generate_v4(),  
  usuario_id UUID NOT NULL REFERENCES auth.users(id) ON DELETE CASCADE,  
  clube_id INTEGER NOT NULL REFERENCES public.clubes(id) ON DELETE CASCADE,  
  membro_id INTEGER REFERENCES public.desbravadores(id) ON DELETE SET NULL,  
  perfil TEXT NOT NULL REFERENCES public.perfis_acesso(codigo),  
  unidade_id INTEGER REFERENCES public.unidades(id) ON DELETE SET NULL,  
  ativo BOOLEAN NOT NULL DEFAULT TRUE  
);
```


Pontuação

```
CREATE TABLE public.pontuacoes (  
  id SERIAL PRIMARY KEY,  
  clube_id INTEGER REFERENCES public.clubes(id),  
  dbv_id INTEGER NOT NULL REFERENCES desbravadores(id),  
  data DATE NOT NULL,  
  presenca BOOLEAN DEFAULT FALSE,  
  pontualidade BOOLEAN DEFAULT FALSE,  
  material BOOLEAN DEFAULT FALSE,  
  uniforme BOOLEAN DEFAULT FALSE,  
  bom_biblia INTEGER DEFAULT 0,  
  pontos_extras INTEGER DEFAULT 0, -- agregado; detalhe em pontuacoes_extras_itens  
  classe_biblica INTEGER DEFAULT 0,  
  especialidade INTEGER DEFAULT 0,  
  pgm_especial INTEGER DEFAULT 0,  
  atividade_unidade INTEGER DEFAULT 0,  
  observacao TEXT,  
  lancado_por TEXT,  
  UNIQUE (dbv_id, data) -- 1 lançamento por membro por dia  
);  
  
-- Detalhe nominal dos pontos extras (migração 083)  
CREATE TABLE public.pontuacoes_extras_itens (  
  id BIGSERIAL PRIMARY KEY,  
  clube_id INTEGER NOT NULL REFERENCES public.clubes(id) ON DELETE CASCADE,  
  dbv_id INTEGER NOT NULL REFERENCES public.desbravadores(id) ON DELETE CASCADE,  
  data DATE NOT NULL,  
  pontos INTEGER NOT NULL,  
  observacao TEXT, -- descrição própria de cada lançamento  
  lancado_por TEXT,  
  created_at TIMESTAMPTZ NOT NULL DEFAULT NOW()  
);
```

Trilha formativa

```
CREATE TABLE public.classes_requisitos_catalogo (  
  id BIGSERIAL PRIMARY KEY,  
  classe_nome TEXT NOT NULL,  
  secao TEXT NOT NULL,  
  secao_ordem INTEGER NOT NULL DEFAULT 1,  
  ordem INTEGER NOT NULL DEFAULT 1,  
  codigo TEXT NOT NULL,  
  codigo_raiz TEXT NOT NULL DEFAULT '', -- hierarquia raiz → subitem  
  subitem TEXT,  
  texto TEXT NOT NULL,  
  avancada BOOLEAN NOT NULL DEFAULT FALSE,  
  pontua BOOLEAN NOT NULL DEFAULT TRUE, -- só raízes contam no percentual  
  especialidade_nome TEXT, -- equivalência automática  
  ativo BOOLEAN NOT NULL DEFAULT TRUE  
);  
  
CREATE UNIQUE INDEX ... ON (classe_nome, secao, codigo, COALESCE(subitem, ''));  
  
CREATE TABLE public.classes_requisitos_progresso (  
  id BIGSERIAL PRIMARY KEY,  
  clube_id INTEGER NOT NULL REFERENCES public.clubes(id) ON DELETE CASCADE,  
  dbv_id INTEGER NOT NULL REFERENCES public.desbravadores(id) ON DELETE CASCADE,  
  requisito_id BIGINT NOT NULL REFERENCES classes_requisitos_catalogo(id),  
  classe_nome TEXT NOT NULL,  
  concluido BOOLEAN NOT NULL DEFAULT TRUE,  
  origem TEXT NOT NULL DEFAULT 'manual'  
    CHECK (origem IN ('manual', 'atividade', 'especialidade')),  
  concluido_por UUID REFERENCES auth.users(id),  
  UNIQUE (clube_id, dbv_id, requisito_id)  
);  
  
-- Estado de validação final (investidura)  
CREATE TABLE public.investidura_itens (  
  id BIGSERIAL PRIMARY KEY,  
  clube_id INTEGER NOT NULL REFERENCES public.clubes(id) ON DELETE CASCADE,  
  dbv_id INTEGER NOT NULL REFERENCES public.desbravadores(id) ON DELETE CASCADE,  
  tipo TEXT NOT NULL CHECK (tipo IN ('classe', 'especialidade')),  
  item_nome TEXT NOT NULL,  
  marcado BOOLEAN NOT NULL DEFAULT TRUE, -- 100% dos requisitos cumpridos  
  entregue BOOLEAN NOT NULL DEFAULT FALSE, -- validado pela diretoria  
  UNIQUE (clube_id, dbv_id, tipo, item_nome)  
);
```

4 Segurança e Controle de Acesso

Quem é quem, o que cada perfil pode fazer, e como isso é imposto em cada camada.

4.1 Defesa em profundidade

O controle de acesso é aplicado em **quatro camadas independentes**. A camada de interface existe para dar boa experiência — ela esconde o que não interessa. A camada que *realmente* protege é o Row Level Security no Postgres: mesmo que alguém chame a API REST diretamente com um token válido, o banco só devolve as linhas dos clubes aos quais aquele usuário está vinculado.

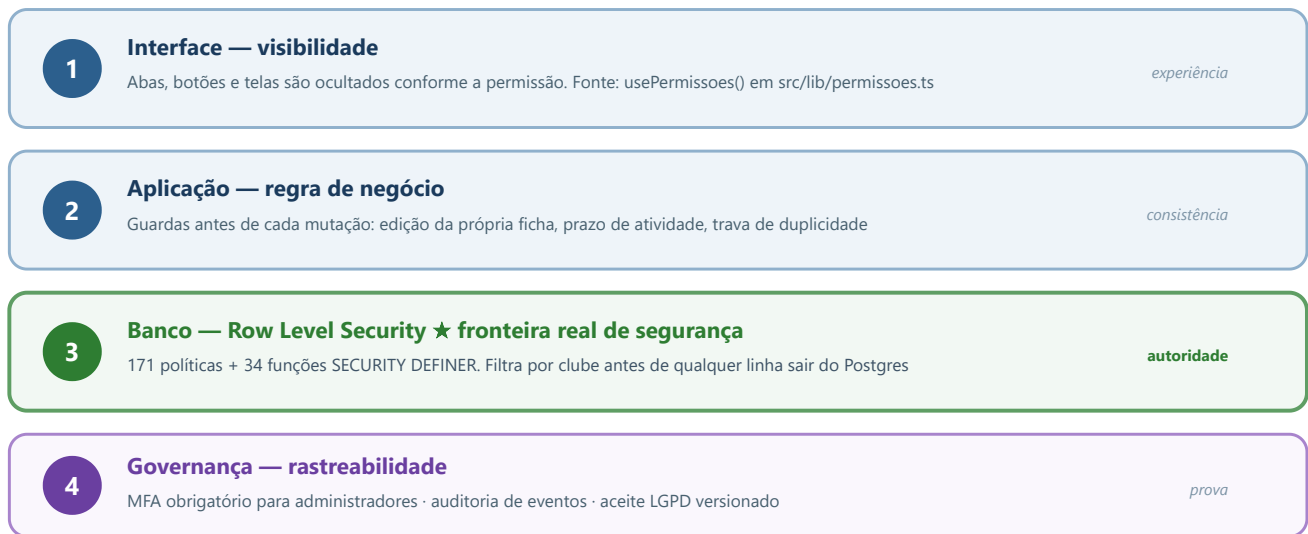


Figura 4.1 — Camadas de controle de acesso. Contornar as camadas 1 e 2 não concede acesso a dado algum.

Regra inegociável

Nenhuma decisão de segurança pode depender *apenas* do cliente. Todo endpoint acessível pelo app é acessível por qualquer cliente HTTP com o mesmo token — por isso cada tabela transacional tem política RLS que verifica o vínculo do usuário com o clube da linha.

4.2 Fluxo de autenticação

A entrada no sistema tem quatro etapas obrigatórias em sequência. Um administrador sem segundo fator configurado é forçado a cadastrá-lo antes de qualquer acesso; um usuário que ainda não aceitou a versão vigente do termo LGPD é bloqueado até aceitá-la.

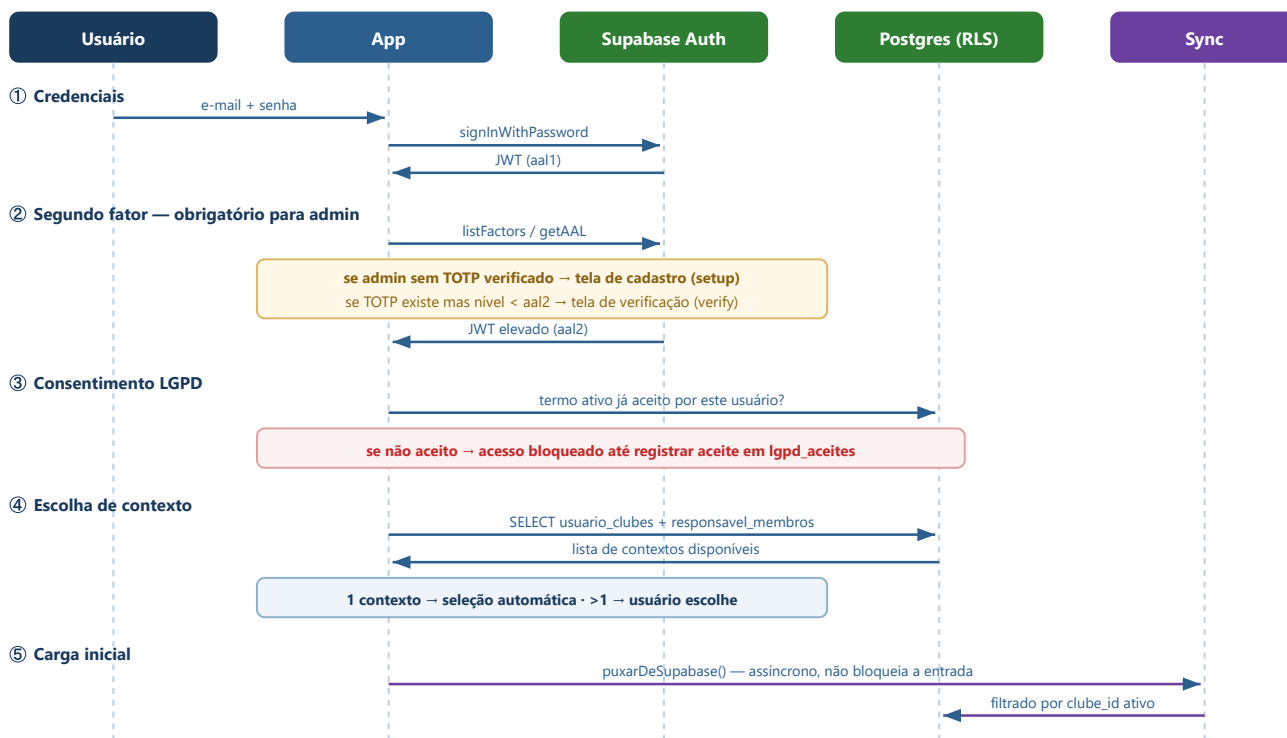


Figura 4.2 — Sequência de autenticação: credencial → MFA → LGPD → contexto → sincronização.

4.3 Modelo de contexto multi-clube

Um **contexto** é a combinação (*usuário, clube, perfil*). Ele é o que determina, a cada momento, quais dados o app carrega e quais telas exibe. Contextos vêm de duas origens:

- **Contexto de clube** — uma linha de `usuario_clubes`. Pode ser restrito a uma unidade (conselheiro) ou a um membro (o próprio desbravador).
- **Contexto de responsável** — derivado de `responsavel_membros`. Dá acesso apenas aos membros tutelados.

União de permissões dentro do mesmo clube

Em `permissoesMescladas()`, o app calcula a união das permissões de **todos os contextos do mesmo clube** do contexto ativo. Assim, alguém que é conselheiro e pai no Clube A enxerga os dois conjuntos de funcionalidade sem precisar alternar. A união **não** atravessa clubes: ser admin no Clube A não confere nada no Clube B.

4.4 Catálogo de perfis

Código	Nome	Escopo	Alcance
<code>admin_ti</code>	Admin TI	plataforma	Todos os clubes, todos os catálogos globais, auditoria integral.
<code>admin_clube</code>	Admin do Clube	clube	Controle administrativo completo de um clube.
<code>usuario_diretoria</code>	Diretoria	clube	Operação ampla: membros, pontuação, unidades, atividades, financeiro.
<code>usuario_secretaria</code>	Secretaria	clube	Cadastros, documentação, atividades, agenda e relatórios.
<code>usuario_tesouraria</code>	Tesouraria	clube	Somente financeiro e relatórios.
<code>usuario_conselheiro</code>	Conselheiro(a)	unidade	Acompanha a sua unidade; lança pontuação e valida classes.
<code>usuario_instrutor</code>	Instrutor(a)	unidade	Igual ao conselheiro, exceto gestão de membros.
<code>usuario_capelao</code>	Capelão	clube	Atividades devocionais, mensagens e relatórios.
<code>usuario_regional</code>	Regional	supra-clube	Exclusivamente validação de classes e especialidades dos clubes vinculados.
<code>usuario_distrital</code>	Distrital	supra-clube	Somente leitura agregada.
<code>usuario_pastor</code>	Pastor	supra-clube	Somente leitura agregada.
<code>usuario_pais</code>	Pais/Responsável	responsável	Somente os membros tutelados, conforme flags em <code>responsavel_membros</code> .
<code>usuario_desbravador</code>	Desbravador	próprio	Somente os próprios dados.
<code>usuarioaventureiro</code>	Aventureiro	próprio	Somente os próprios dados.

Perfis legados e sua normalização

Contas anteriores à migração multi-clube ainda podem carregar códigos antigos. A função `normalizarPerfil()` os traduz de forma transparente antes de qualquer verificação:

Código legado	Equivalente atual
<code>admin_total</code>	<code>admin_ti</code>
<code>admin_geral</code>	<code>admin_clube</code>
<code>admin_diretoria</code>	<code>usuario_diretoria</code>
<code>desbravador</code>	<code>usuario_desbravador</code>

4.5 Matriz de permissões

As 15 permissões atômicas do sistema, definidas em `src/lib/permissions.ts`. Esta matriz é a **fonte única de verdade** sobre o que cada perfil pode fazer na interface.

Permissão	Admin TI	Admin Clube	Diretoria	Secretaria	Tesouraria	Conselheiro	Instrutor	Capelão	Regional	Distr./Pastor	Pais	Membro
admin_plataforma	•	—	—	—	—	—	—	—	—	—	—	—
admin_clube	•	•	—	—	—	—	—	—	—	—	—	—
gerenciar_clubes	•	—	—	—	—	—	—	—	—	—	—	—
gerenciar_acessos	•	•	—	—	—	—	—	—	—	—	—	—
gerenciar_membros	•	•	•	•	—	—	—	—	—	—	—	—
gerenciar_documentos	—	—	—	•	—	—	—	—	—	—	—	—
gerenciar_pontuacao	•	•	•	—	—	•	•	—	—	—	—	—
gerenciar_unidades	•	•	•	—	—	—	—	—	—	—	—	—
gerenciar_agenda	•	•	•	•	—	•	•	—	—	—	—	—
gerenciar_atividades	•	•	•	•	—	•	•	•	—	—	—	—
enviar_mensagens	•	•	•	•	—	—	—	•	—	—	—	—
validar_classes	•	•	•	•	—	•	•	—	•	—	—	—
ver_relatorios	•	•	•	•	•	•	•	•	—	•	—	—
ver_financeiro	•	•	•	—	•	—	—	—	—	—	—	—
ver_unidade	•	•	•	•	—	•	•	•	—	•	—	—
ver_filhos	•	•	—	—	—	—	—	—	—	—	•	—

Leitura importante da matriz

Desbravador e Aventureiro têm matriz vazia. Isso é intencional: eles não recebem nenhuma permissão administrativa. O acesso aos próprios dados não é dado por permissão, e sim pelo vínculo `membro_id` no contexto, combinado com as políticas RLS que liberam a leitura da própria linha.

Secretaria não lança pontuação e Conselheiro/Instrutor não enviam mensagens — são separações deliberadas de responsabilidade, não omissões.

Instrutor difere de Conselheiro em um único ponto: não possui `gerenciar_membros`. Ele acompanha e valida a trilha formativa, mas não edita fichas de terceiros.

4.6 Matriz de perfis × telas

Traduzindo a matriz anterior para o que o usuário efetivamente vê. Cada linha indica a permissão que destrava a tela e quais perfis a alcançam.

Tela / rota	Permissão exigida	Perfis com acesso
Início · Ranking · Classes	livre	Todos os autenticados (o conteúdo é filtrado pelo escopo).
/pontuacao , /extras	gerenciar_pontuacao	Admin TI, Admin Clube, Diretoria, Conselheiro, Instrutor
/membros , /membro/[id]	gerenciar_membros	Admin TI, Admin Clube, Diretoria, Secretaria
/unidades	gerenciar_unidades	Admin TI, Admin Clube, Diretoria
/calendario	gerenciar_agenda	Admin TI, Admin Clube, Diretoria, Secretaria, Conselheiro, Instrutor
/atividades	gerenciar_atividades	Os acima + Capelão
/mensagens , /admin/mensagens	enviar_mensagens	Admin TI, Admin Clube, Diretoria, Secretaria, Capelão
/relatorios	ver_relatorios	Todos os perfis administrativos + Tesouraria, Distrital, Pastor
/classes/enviar	gerenciar_atividades	Perfis com gestão de atividades
/admin/clubes , /admin/ranking-clubes , /admin/classificacao	gerenciar_clubes	Somente Admin TI
/admin/acessos , /admin/vincular- usuarios , /admin/menus- publicos	gerenciar_acessos	Admin TI, Admin Clube
/admin/auditoria	admin_plataforma U admin_clube U gerenciar_acessos	Admin TI (todos os clubes), Admin Clube (o próprio clube)
/admin/lgpd	gerenciar_acessos U admin_clube	Admin TI, Admin Clube
/admin/aparencia	perfil ∈ {admin_ti, admin_clube}	Admin TI, Admin Clube
/admin/pre-cadastros	gerenciar_membros U admin_clube	Admin TI, Admin Clube, Diretoria, Secretaria
/admin/modelos	admin_clube U gerenciar_pontuacao U gerenciar_documentos	Admin TI, Admin Clube, Diretoria, Secretaria, Conselheiro, Instrutor
/admin/regionais	admin_plataforma U gerenciar_acessos	Admin TI, Admin Clube
/classes/catalogo , /classes/requisitos , /especialidades/catalogo	perfil ∈ {admin_ti, admin_total}	Somente Admin TI — são catálogos globais compartilhados
/importar	gerenciar_membros U gerenciar_agenda U gerenciar_pontuacao	Perfis operacionais do clube
/pre-cadastro/[token] , /convite/[token]	público	Sem autenticação — acesso por token

Catálogos globais só para Admin TI

As telas de catálogo de classes, requisitos e especialidades editam dados **compartilhados por todos os clubes**. Por isso o gate é por perfil explícito, e não por permissão: uma alteração ali afeta a base inteira, não apenas um clube.

4.7 Row Level Security no Postgres

Todas as tabelas do esquema `public` têm RLS habilitado, totalizando 171 políticas distintas. Os padrões recorrentes são três:

Padrão A — Isolamento por clube (o mais comum)

```
-- Aplica-se a pontuacoes, desbravadores, documentos, atividades, unidades...
CREATE POLICY "tabela_select_clube" ON public.tabela
  FOR SELECT USING ( public.current_user_has_clube(clube_id) );

CREATE POLICY "tabela_write_clube" ON public.tabela
  FOR ALL
  USING      ( public.current_user_can_manage_pontuacao(clube_id) )
  WITH CHECK ( public.current_user_can_manage_pontuacao(clube_id) );
```

Padrão B — Catálogo global (leitura aberta, escrita restrita)

```
-- classes_requisitos_catalogo, especialidades_modelo, classes_modelo...
CREATE POLICY "catalogo_select_authenticated" ON public.catalogo
  FOR SELECT USING ( auth.role() = 'authenticated' );

CREATE POLICY "catalogo_admin_all" ON public.catalogo
  FOR ALL
  USING      ( public.current_user_is_admin_ti() )
  WITH CHECK ( public.current_user_is_admin_ti() );
```

Padrão C — Dado próprio ou tutelado

```
-- Membro vê a si mesmo; responsável vê quem tutela; gestor vê o clube
CREATE POLICY "membro_select proprio ou tutelado" ON public.desbravadores
  FOR SELECT USING (
    public.current_user_has_clube(clube_id)
    OR id = public.current_user_dbv_id()
    OR public.current_user_is_responsavel_membro(id)
  );
```

Por que funções em vez de subconsultas inline

Encapsular a lógica em funções `STABLE SECURITY DEFINER` traz três ganhos: **(1)** evita recursão infinita de RLS (a função roda com o dono da tabela, ignorando as políticas da tabela que está consultando); **(2)** permite ao planejador cachear o resultado dentro da mesma consulta; **(3)** centraliza a regra — mudar quem é administrador exige alterar uma função, não 171 políticas.

4.8 Funções de segurança (SECURITY DEFINER)

Função	Retorno	Regra que implementa
<code>current_user_is_admin_ti()</code>	boolean	Verdadeiro se há vínculo ativo com perfil <code>admin_ti</code> em <code>usuario_clubes</code> ou se a linha legada em <code>usuarios</code> tem perfil <code>admin_ti</code> / <code>admin_total</code> .
<code>current_user_has_clube(clube_id)</code>	boolean	Verdadeiro para Admin TI (acesso universal) ou se existe vínculo ativo com aquele clube. É o predicado base do isolamento multi-tenant.
<code>current_user_perfil()</code>	text	Perfil legado da tabela <code>usuarios</code> .
<code>current_user_dbv_id()</code>	integer	Membro vinculado à conta — permite ao desbravador ler a própria linha.
<code>current_user_unidade_id()</code>	integer	Unidade vinculada — usada em recortes de conselheiro.
<code>is_admin()</code>	boolean	Amplo: cobre perfis administrativos e operacionais (diretoria, secretaria, conselheiro, instrutor), nas duas fontes de verdade.
<code>current_user_can_admin_clube(clube_id)</code>	boolean	Administração plena daquele clube específico.
<code>current_user_can_manage_pontuacao(clube_id)</code>	boolean	Restrito a <code>admin_clube</code> e <code>usuario_diretoria</code> no clube alvo (além de Admin TI).
<code>current_user_can_manage_atividades(clube_id)</code>	boolean	Quem pode criar e avaliar atividades no clube.
<code>current_user_can_manage_docs_clube(clube_id)</code>	boolean	Gestão documental no clube.
<code>current_user_is_responsavel_membro(membro_id)</code>	boolean	Verdadeiro se existe tutela ativa com <code>pode_visualizar = TRUE</code> .
<code>current_user_can_parent_edit_docs(membro_id)</code>	boolean	Cruza a tutela com <code>documentos_pais_config</code> para decidir se o responsável pode enviar documentos.
<code>current_user_can_view_doc_files(...)</code>	boolean	Autoriza a leitura de arquivos no Storage.
<code>current_user_can_manage_member_photo(...)</code>	boolean	Autoriza troca de foto do membro.
<code>current_user_pode_editar_ficha_basica(...)</code>	boolean	Regra fina de autoedição: conselheiro/instrutor editam a própria ficha; pais sempre podem editar a do filho.

4.9 Storage e políticas de arquivos

Bucket	Conteúdo	Controle de acesso
<code>fotos_membros</code>	Foto de perfil dos membros	Leitura por quem tem vínculo com o clube; escrita validada por <code>current_user_can_manage_member_photo()</code> .
<code>documentos_fotos</code>	Anexos documentais (RG, ficha de saúde, autorizações)	Dado sensível. Leitura por <code>current_user_can_view_doc_files()</code> ; responsáveis dependem de <code>documentos_pais_config</code> .
<code>atividades</code>	Anexos de atividades, materiais de plano formativo e insígnias de especialidade	Bucket público de leitura; escrita restrita a quem gerencia atividades.

Nota operacional sobre uploads

O envio de imagens força `contentType: image/jpeg` em vez de confiar no tipo relatado pelo seletor de arquivos. Isso resolve rejeições do bucket quando o dispositivo informa um tipo vazio ou não suportado (HEIC do iPhone, por exemplo).

5 Módulos Funcionais

Cada módulo detalhado: o que faz, como faz, quais tabelas usa e quem pode operá-lo.

5.1 Membros e unidades

A ficha do membro (`app/membro/[id].tsx`) é a tela mais densa do sistema, organizada em seis abas: **Documentos**, **Classes**, **Especialidades**, **Receber** (itens pendentes de validação), **Responsáveis** e **Editar**.

Regras de negócio

Regra	Comportamento
Cargo por programa	A lista de cargos vem de <code>cargos_modelo</code> filtrada pelo programa do clube ativo. Clube de Aventureiros não oferece “Desbravador”, e vice-versa.
Cargo por idade	<code>cargoBloqueadoPorIdade()</code> impede atribuir cargo de diretoria (mínimo 16 anos) a menores, e cargos juvenis a adultos. Ao mudar a data de nascimento, o cargo incompatível é limpo automaticamente.
Gênero e nomenclatura	Cada cargo tem forma masculina e feminina; <code>adaptarCargo()</code> converte ao alternar o gênero.
Cargo adicional	Um membro pode acumular função (ex.: Desbravador + Capitão de unidade) — coluna <code>cargo_adicional</code> .
Inativação	Preferida à exclusão: <code>ativo = false</code> preserva pontuação, classes e histórico. A exclusão física é reservada a registros criados por engano.
Autoedição	Conselheiro e instrutor editam a própria ficha mesmo sem <code>gerenciar_membros</code> ; responsáveis sempre podem editar a ficha do filho tutelado (migrações 080 e 081).
Unidade como texto livre	Não há catálogo fechado de unidades. Para evitar mistura entre programas, a criação/edição bloqueia nomes que coincidam com classes do <i>outro</i> programa.

Vinculação de login

Um membro pode ter conta de acesso. O vínculo é feito pela RPC `atualizar_login_membro` (migração 057), que cria ou atualiza a linha em `usuario_clubes` com `membro_id` preenchido — é isso que faz o app reconhecer “esta conta é este desbravador”.

5.2 Pontuação

A tela de pontuação (`app/(tabs)/pontuacao.tsx`) opera por data: escolhe-se o dia e lançam-se os itens para todos os membros presentes.

Fórmula de cálculo

Cada item padrão tem duas representações: um **booleano** (marcou ou não) e um **valor gravado** (`*_pts`). O valor gravado tem precedência — assim, alterar a configuração do clube no futuro *não reescreve o passado*.

```

-- Total de um lançamento diário (src/stores/pontuacaoStore.ts → calcSQL)
total_dia =
  COALESCE(presenca_pts,      presenca      × cfg.presenca,      0)
+ COALESCE(pontualidade_pts, pontualidade × cfg.pontualidade, 0)
+ COALESCE(material_pts,     material     × cfg.material,     0)
+ COALESCE(uniforme_pts,     uniforme     × cfg.uniforme,     0)
+ COALESCE(pontos_extras, 0)

-- Total acumulado do membro
total_membro = Σ total_dia + Σ pontuacoes_custom.pontos

-- Ranking de unidade: soma dos membros (com fator de escala) + pontos diretos
total_unidade = ( Σ total_membro dos integrantes ) × 0,015
                + Σ pontuacoes_unidades.pontos

```

Por que o fator 0,015 no ranking de unidade

Sem o fator, o total agregado dos membros dominaria completamente a nota da unidade, tornando irrelevantes os pontos atribuídos ao coletivo. O fator reduz a soma individual à mesma ordem de grandeza das pontuações diretas, equilibrando desempenho individual e desempenho coletivo.

Valores padrão e itens configuráveis

Item padrão	Valor default	Origem
Presença	25	<code>config_pontuacao</code> por clube; ajustável na administração.
Pontualidade	100	
Material	25	
Uniforme	25	

Além desses, cada clube define **itens próprios** em `pontuacao_itens` (título, sigla e valor). Os lançamentos vão para `pontuacoes_custom`, uma linha por item — o que preserva o nome de cada item no extrato do membro.

Quem pode lançar

Interface: `gerenciar_pontuacao` — Admin TI, Admin do Clube, Diretoria, Conselheiro e Instrutor. No banco, a política é **mais restritiva**: `current_user_can_manage_pontuacao()` aceita apenas `admin_clube` e `usuario_diretoria` (além de Admin TI). Conselheiro e instrutor lançam pela tela operando sob as políticas gerais do clube.

5.3 Pontos extras e descontos

Pontos extras são lançamentos com **descrição livre** — “Ano Bíblico”, “Calebe”, “Ajudou na cozinha”. A tela `app/(tabs)/extras.tsx` tem duas abas: *Adicionar* (lançamento em lote para vários membros) e *Histórico* (consulta, edição e exclusão).

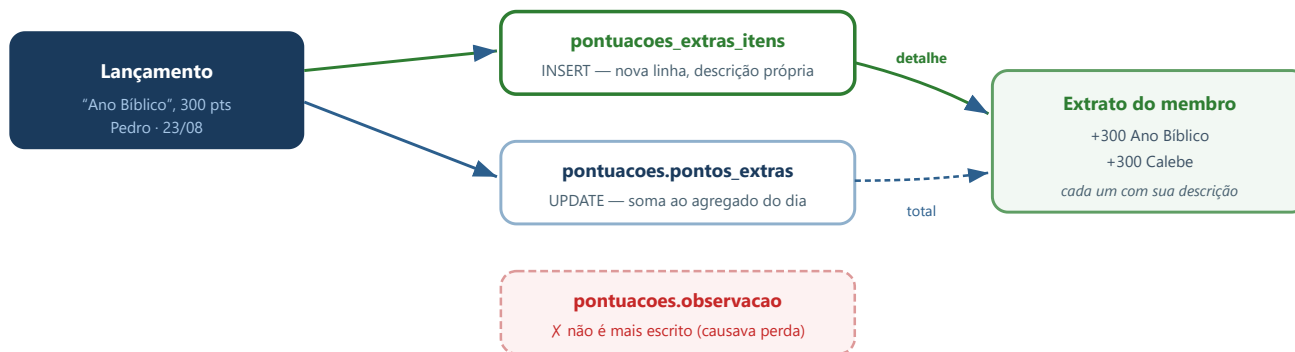


Figura 5.1 — Fluxo de um ponto extra: a descrição vai para a tabela de detalhe, o valor soma no agregado.

Histórico: entradas legadas

Lançamentos feitos *antes* da migração 083 existem apenas como agregado, sem detalhe por descrição. A tela de Histórico os identifica como origem `legado` e os exibe como uma única linha combinada, usando `-pontuacoes.id` como identificador (negativo) para não colidir com os ids positivos da tabela de detalhe. Essas entradas **não podem ser reconstituídas** — não havia trilha de auditoria para esse fluxo à época.

Descontos

Um desconto é um ponto extra com valor **negativo**, lançado pela tela de pontuação (`adicionarPontosExtras(ids, data, -valor, motivo, usuario)`). Usa exatamente o mesmo caminho de código, portanto herda o mesmo tratamento de descrição individual.

5.4 Ranking

Visão	Critério
Individual	Soma de todos os lançamentos do membro, com recorte opcional por grupo (desbravadores, diretoria, conselheiros).
Por unidade	Soma dos membros $\times 0,015$ + pontuações diretas da unidade. As pseudo-unidades "Diretoria" e "Sem unidade" são excluídas.
Entre clubes	Módulo administrativo com requisitos avaliados, notas por clube e níveis com estrelas (<code>ranking_clubes_*</code>).

O **extrato individual** (`app/extrato/[dbv_id].tsx`) detalha dia a dia cada linha que compõe o total, com navegação direta para a tela de origem do lançamento.

5.5 Classes e requisitos

O catálogo modela cada requisito oficial com hierarquia própria. Um requisito-raiz pode ter subitens; apenas as raízes (`pontua = true`) entram no cálculo de percentual — os subitens são detalhamento.

Organização em três trilhas

Trilha	Composição	Regra de desbloqueio
Regulares	Amigo, Companheiro, Pesquisador, Pioneiro, Excursionista, Guia — cada uma com sua avançada.	Sempre visíveis. As avançadas liberam para marcação com > 15 anos ou com as 6 básicas concluídas.
Agrupadas	Mesmas 6 classes, em versão condensada por faixa etária (sufixo <code>- Agrupadas</code>).	Requisitos filtrados por <code>idade_agrupada_min/max</code> contra a idade real do membro.
Liderança	Líder, Líder Máster e Líder Máster Avançada.	Sequencial: Líder exige as 6 básicas (ou > 15 anos); Líder Máster exige Líder; a avançada exige Líder Máster.

Caminhos combináveis

`classesAvancadoDesbloqueado()` aceita **mistura de trilhas**: concluir Amigo pelas Agrupadas e Companheiro pelas Regulares conta igualmente para as “6 básicas completas”. Isso reflete a realidade dos clubes, onde membros entram em idades diferentes.

Formas de conclusão de um requisito

Origem	Gatilho	Mecanismo
<code>manual</code>	Marcação pela liderança	Insert direto em <code>classes_requisitos_progresso</code> com <code>concluido_por</code> registrado.
<code>atividade</code>	Aprovação de atividade vinculada a um plano formativo	Trigger propaga a conclusão para os requisitos do plano.
<code>especialidade</code>	Membro conquista uma especialidade	<code>sync_requisitos_por_especialidade</code> compara <code>especialidade_nome</code> do catálogo, sem acento e sem caixa, e marca o requisito e a sua raiz.

Requisitos compartilhados

Requisitos que se repetem entre classes (ex.: “Ser membro ativo do clube”) carregam a mesma `chave_compartilhada`. O trigger `replicar_requisito_compartilhado` propaga a conclusão para todas as ocorrências, evitando retrabalho.

Formatos de resposta

A coluna `formato_resposta` define a interação: `nenhum` (só marcar), `texto` (dissertativa em `classes_requisitos_respostas`), `upload` (arquivos em `classes_requisitos_arquivos`, limitados por `max_arquivos`), `texto_upload` (ambos) e `checkbox`.

5.6 Especialidades

O catálogo global (`especialidades_modelo`) organiza mais de mil especialidades por categoria e subcategoria, com insígnia e requisitos oficiais. As conquistas do membro ficam em `especialidades`.

Origem da conquista

A função `origemDaEspecialidade()` distingue duas procedências, exibidas como etiqueta na ficha:

- Automática** — decorrente de atividade aprovada, com referência à atividade de origem preservada mesmo se ela for excluída depois (`atividade_origem_excluida`).
- Manual** — marcada pela liderança, registrando `marcado_por_nome` e `marcado_em`.

Trava contra duplicidade

A tabela possui `UNIQUE (dbv_id, nome)`, restaurada explicitamente na migração 086 — a restrição original nunca chegou a ser criada em bancos onde a tabela já existia antes da migração 001 (efeito de `CREATE TABLE IF NOT EXISTS`). Complementarmente, o app normaliza o nome antes de gravar (colapsa espaços múltiplos) e compara sem acento e sem caixa ao verificar se a especialidade já foi conquistada.

5.7 Investidura e validação

`investidura_itens` guarda dois estados independentes por item formativo:

Campo	Significado
<code>marcado</code>	Calculado pelo sistema: 100% dos requisitos que pontuam foram concluídos. Aparece na aba “Receber” da ficha.
<code>entregue</code>	Ato humano: a diretoria confirmou a entrega/validação. É terminal — a função <code>recalcular_classe_receber</code> não mexe mais no item depois disso.

A tela `app/admin/aprovacoes.tsx` consolida, para todo o clube, os itens que atingiram 100% e aguardam validação, permitindo aprovar em lote sem abrir ficha por ficha.

5.8 Documentos

Cada clube define quais documentos exige, via `documentos_modelo` — um registro com `clube_id NULL` é padrão do programa; um com `clube_id` preenchido sobrescreve para aquele clube. Os quinze campos padrão vão de RG e CPF a autorização de viagem e antecedentes criminais.

Aspecto	Comportamento
Status por campo	<code>OK</code> (entregue), <code>NOK</code> (pendente) ou <code>NA</code> (não se aplica).
Anexos	Até 3 arquivos por campo (migração 023), armazenados em <code>documentos_fotos</code> .
Contabilização	Um campo só conta como entregue se estiver <code>NA</code> , ou <code>OK</code> com anexo — evita marcar como pronto sem comprovação.
Acesso do responsável	Governado por <code>documentos_pais_config</code> : por clube, define quais campos o responsável vê e quais pode enviar.
Requisito ↔ documento	<code>sync_requisito_documento_identidade</code> conclui automaticamente o requisito de classe correspondente quando o documento de identidade é entregue.

5.9 Atividades e planos formativos

Uma atividade é uma tarefa avaliativa com prazo. O ciclo de vida da *resposta* de cada membro segue a máquina de estados abaixo.

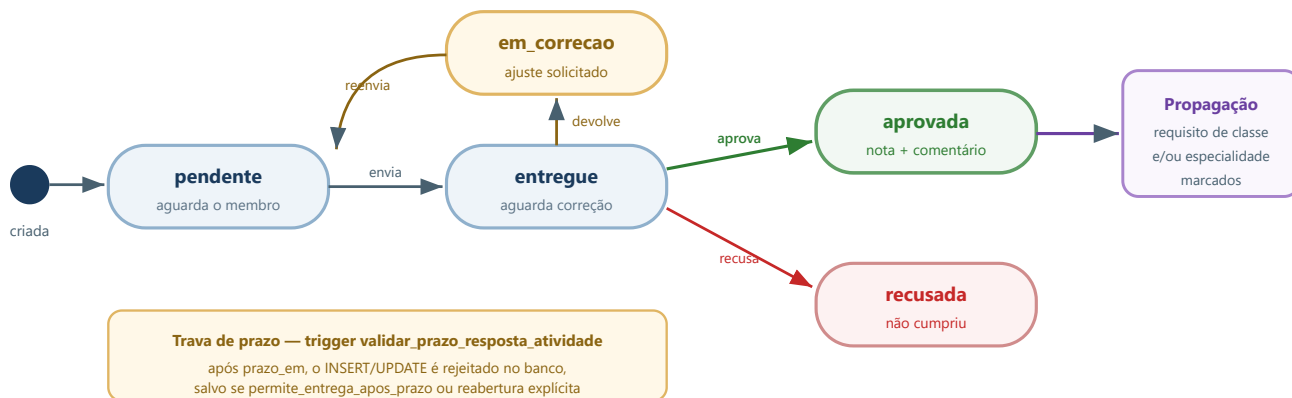


Figura 5.2 — Máquina de estados da resposta a uma atividade avaliativa.

Recurso	Detalhe
Segmentação	<code>destino</code> = <code>todos</code> <code>unidade</code> <code>membro</code> ; destinatários explícitos em <code>atividades_alvos</code> .
Prazo com timezone	<code>prazo_em</code> é <code>timestampz</code> (migração 042) — evita divergência de fuso entre servidor e dispositivo.
Trava de prazo	Imposta por trigger no banco (migração 033), não apenas na interface.
Reabertura	Migrações 043 e 044: o avaliador reabre uma resposta já avaliada, com novo prazo, registrando quem reabriu.
Conversa	<code>atividades_mensagens</code> mantém o diálogo entre avaliador e membro dentro da atividade.
Tema visual	Cor e ícone por atividade (migrações 039 e 040), com paleta configurável por clube.
Plano formativo	Agrupa N atividades que juntas concluem uma classe ou especialidade; <code>avaliacoes_necessarias</code> define quantas aprovações são exigidas.

5.10 Comunicação

Canal	Funcionamento
Mural do clube	<code>mensagens_clube</code> com título e corpo. <code>mensagens_clube_lidos</code> registra a leitura por usuário e <code>_ocultos</code> permite que cada um remova a mensagem da sua visão sem apagá-la para os demais.
Notificações push	Tokens Expo em <code>push_tokens</code> . Disparadas em eventos relevantes — lançamento de pontos extras, nova atividade, avaliação concluída.
WhatsApp	<code>whatsapp_fila</code> acumula mensagens para envio por integração externa; <code>whatsapp_config</code> guarda os parâmetros por clube.

5.11 Calendário

`eventos` registra a programação do clube: data, horário, local, atividade, responsável, apoio, material necessário e observações, segmentada por semestre. Gerido por `gerenciar_agenda` .

5.12 Pré-cadastro público

Permite que famílias se inscrevam sem ter conta. O clube gera um link tokenizado (`pre_cadastro_links`) e o divulga; o formulário é acessível sem autenticação.

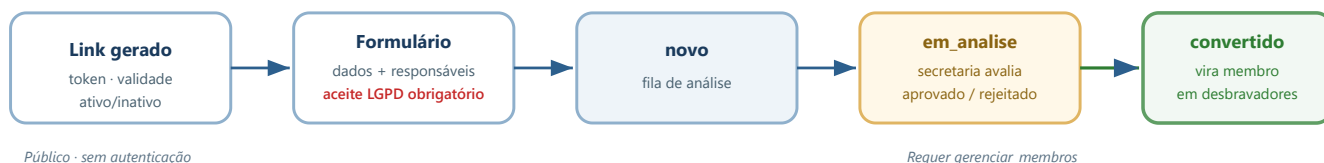


Figura 5.3 — Ciclo do pré-cadastro público até a conversão em membro.

Convite de responsável

Fluxo paralelo: a secretaria gera um convite (`responsavel_convites`) com token e telefone. O responsável acessa `/convite/[token]`, cria a conta e a RPC `aceitar_convite_responsavel` materializa o vínculo em `responsavel_membros` e o contexto em `usuario_clubes`.

5.13 Relatórios e importação

Recurso	Detalhe
Relatórios	Emissão em PDF via <code>expo-print</code> , incluindo progresso de classes e situação documental. Exige <code>ver_relatorios</code> .
Importação	Leitura de planilhas com a biblioteca <code>xlsx</code> para carga em massa de membros, eventos e pontuação. Cada execução é rastreada em <code>importacoes_lote</code> e seus itens.
Classificação SGC	Alinhamento com a nomenclatura oficial do Sistema de Gestão de Clubes (migração 036).

5.14 Administração da plataforma

Tela	Alcance	Função
<code>/admin/clubes</code>	Admin TI	Criação e configuração de clubes; dispara <code>onboard_clube</code> para clonar os modelos iniciais.
<code>/admin/acessos</code>	Admin Clube	Concessão e revogação de perfis; usa a RPC <code>gerenciar_acesso_usuario</code> .
<code>/admin/vincular-usuarios</code>	Admin Clube	Associa contas existentes a membros do clube.
<code>/admin/aprovacoes</code>	Clube	Validação em lote de classes e especialidades que atingiram 100%.
<code>/admin/auditoria</code>	Admin TI / Clube	Consulta da trilha de eventos, com estado antes/depois.
<code>/admin/lgpd</code>	Admin Clube	Publicação de novas versões do termo e consulta dos aceites.
<code>/admin/aparencia</code>	Admin Clube	Cores, logo e paleta de atividades do clube.
<code>/admin/modelos</code>	Clube	Catálogos do clube: itens de pontuação, campos documentais, cargos.
<code>/admin/pre-cadastros</code>	Clube	Triagem das submissões públicas e conversão em membro.
<code>/admin/regionais</code>	Admin TI	Estrutura de regionais e distritos.
<code>/admin/ranking-clubes</code> , <code>/admin/classificacao</code>	Admin TI	Requisitos, notas e níveis do ranking entre clubes.
<code>/admin/menus-publicos</code>	Admin Clube	Configuração do que aparece sem autenticação.
<code>/admin/formativos</code>	Clube	Planos formativos que ligam atividades a classes e especialidades.

6 Sincronização Offline-First

Como o aplicativo continua útil sem rede — e reconcilia tudo quando a rede volta.

6.1 Estratégia local-first

No Android, o app mantém um banco SQLite com 23 tabelas espelhando o subconjunto operacional do Postgres. Toda escrita acontece **primeiro no banco local** e é enfileirada para envio. A interface responde imediatamente; a rede é um detalhe de fundo.

Na Web não há SQLite — as operações vão diretamente ao Supabase. Por isso a maior parte das funções do sistema tem dois caminhos, selecionados por `Platform.OS === 'web'`.

Aspecto	Android (nativo)	Web (PWA)
Persistência	SQLite via <code>expo-sqlite</code>	Sem banco local; cache em memória
Escrita	Local → <code>fila_sync</code> → envio assíncrono	Direto no Supabase
Leitura offline	Completa	Indisponível
Conflitos	Resolvidos no envio da fila	Não se aplicam

6.2 Fluxo de descida (pull)

`puxarDeSupabase()` orquestra cinco descidas independentes, sempre filtradas pelo `clube_id` ativo:

Função	Escopo
<code>puxarMembros()</code>	Unidades e membros — destrava quase todas as telas, por isso é a primeira.
<code>puxarPontuacoes()</code>	Configuração, itens, lançamentos, extras itemizados e pontuação de unidade.
<code>puxarClassesEspecialidades()</code>	Progresso de classes e especialidades conquistadas.
<code>puxarComunicacao()</code>	Mural e eventos do calendário.
<code>puxarDocumentos()</code>	Status documental e referências de anexos.
<code>puxarAtividades()</code>	Atividades, alvos, respostas, anexos e mensagens.

Como o PostgREST limita a resposta a 1.000 linhas por padrão, o helper `buscarTudo()` pagina até que um lote venha menor que o tamanho da página — indispensável para o catálogo de requisitos, que ultrapassa esse limite.

Incidente corrigido: vazamento entre clubes

As descidas de `unidades` e `desbravadores` originalmente não filtravam por `clube_id` — como a política RLS dessas tabelas é permissiva para quem tem múltiplos vínculos, o dispositivo baixava dados de *todos* os clubes acessíveis e os misturava. Pior: a rotina que deriva unidades a partir do campo `unidade_nome` dos membros recriava localmente unidades do outro programa, mesmo após a correção da consulta de unidades. A solução exigiu **três** ajustes: filtrar as unidades, filtrar os membros, e passar a remover órfãos também na tabela de unidades.

6.3 Fluxo de subida (push)

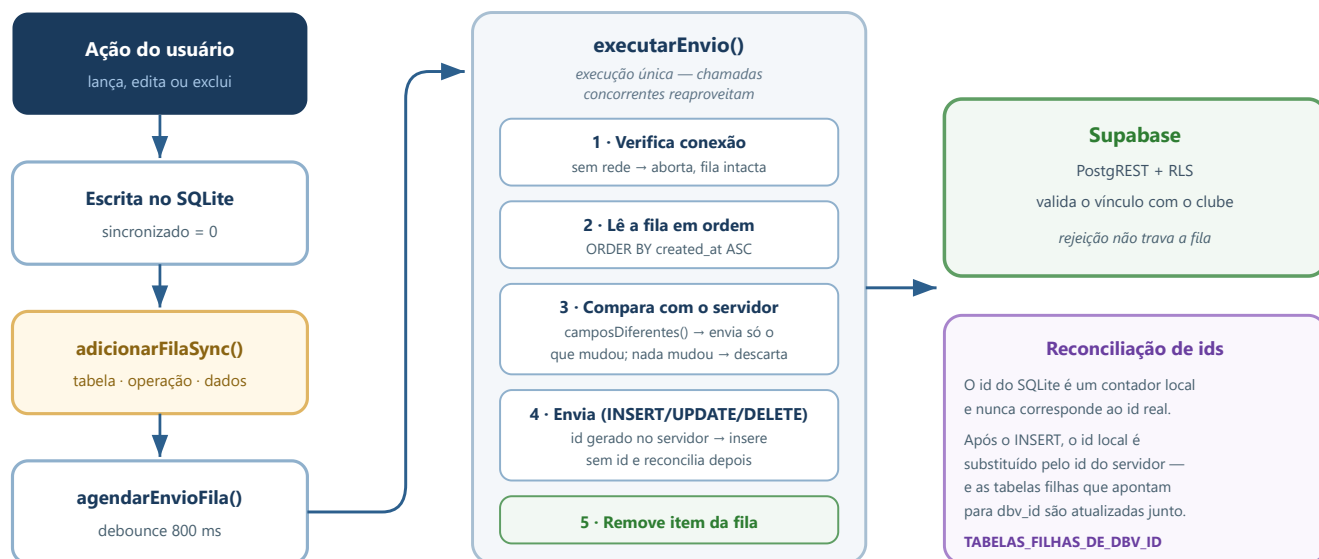


Figura 6.1 — Ciclo completo de subida, da ação do usuário à reconciliação de identificadores.

6.4 Reconciliação de identificadores

É o ponto mais delicado da sincronização. Tabelas com id `SERIAL` / `BIGSERIAL` no Postgres têm o id atribuído **pelo servidor**; o `AUTOINCREMENT` do SQLite é apenas um contador do aparelho. Enviar o id local em um upsert sobrescreveria a linha errada — ou falharia e travaria a fila permanentemente.

```
// src/lib/sync.ts
const TABELAS_ID_GERADO_NO_SERVIDOR = new Set([
  'pontuacoes', 'pontuacoes_custom', 'pontuacoes_extras_itens',
  'pontuacoes_unidades', 'config_pontuacao_itens',
  'mensagens_clube', 'desbravadores',
]);

// Ao reconciliar o id de um membro, as filhas precisam acompanhar
const TABELAS_FILHAS_DE_DBV_ID = [
  'documentos', 'progresso_classes', 'especialidades',
  'pontuacoes', 'pontuacoes_custom',
];
```

Para essas tabelas o INSERT é enviado **sem id**; assim que o Supabase responde com o id real, o registro local é atualizado e todas as referências filhas são corrigidas na mesma transação.

6.5 Remoção de órfãos

Um `INSERT OR REPLACE` grava e atualiza, mas nunca apaga. Sem tratamento, uma linha excluída no servidor — ou que chegou ao dispositivo por engano — permaneceria no aparelho para sempre. `removerOrfaos()` resolve isso a cada descida completa:

1. Coleta os ids presentes na resposta do servidor.
2. Coleta os ids ainda pendentes em `fila_sync` — **preservados**, pois são criações locais que o servidor ainda não conhece.

3. Apaga localmente tudo que não está em nenhum dos dois conjuntos.

O passo 2 é o que evita perda de dados

Sem preservar os pendentes, um lançamento feito offline seria apagado pela primeira sincronização antes mesmo de ser enviado.

6.6 Resolução de conflitos

Situação	Tratamento
Mesmo campo alterado nos dois lados	<i>Last-write-wins</i> pela ordem de chegada da fila. Aceitável no domínio: um lançamento é feito por uma pessoa por vez.
Alteração local idêntica ao servidor	<code>camposDiferentes()</code> retorna vazio e o item é descartado sem tráfego de rede.
Divergência de tipo entre SQLite e Postgres	<code>equivalente()</code> normaliza booleano ↔ 0/1, número ↔ texto, e <code>NULL</code> ↔ string vazia antes de comparar.
Linha excluída no servidor	O UPDATE não encontra alvo; o item é removido da fila e a linha local sai na próxima limpeza de órfãos.
Rejeição por RLS	O erro é registrado, o item sai da fila e a sincronização prossegue — um item inválido não pode bloquear os demais.

7 Funções, Triggers e Automações de Banco

Regra de negócio que vive no Postgres — válida para qualquer cliente, inclusive fora do app.

7.1 Catálogo de funções

São 34 funções. As de segurança estão na seção 4.8; abaixo, as de domínio e operação.

Automação da trilha formativa

Função	Comportamento
<code>sync_requisitos_por_especialidade()</code>	Ao registrar uma especialidade com status <code>OK</code> , localiza no catálogo os requisitos cujo <code>especialidade_nome</code> corresponde (comparação sem acento e sem caixa) e marca tanto o requisito quanto a sua raiz — pois requisitos do tipo “complete uma das seguintes especialidades” são satisfeitos por qualquer uma da lista.
<code>sync_especialidade_por_requisito()</code>	Caminho inverso: concluir manualmente um requisito que representa uma especialidade cria a conquista correspondente.
<code>replicar_requisito_compartilhado()</code>	Propaga a conclusão para todas as ocorrências que compartilham a mesma <code>chave_compartilhada</code> em classes diferentes.
<code>sync_requisito_por_resposta()</code>	Marca o requisito como concluído quando uma resposta válida é registrada.
<code>sync_requisito_documento_identidade()</code>	Conclui o requisito de documento de identidade quando o documento correspondente é entregue.
<code>recalcular_classe_receber()</code>	Recalcula o percentual da classe. Ao atingir 100%, cria o item em <code>investidura_itens</code> com <code>marcado = TRUE</code> . Não reabre item já entregue — validação da diretoria é definitiva.
<code>marcar_classe_completa()</code>	Marca de uma vez todos os requisitos de uma classe (uso administrativo, para migração de histórico).
<code>unaccent_simples()</code>	Remove acentuação sem depender da extensão <code>unaccent</code> , indisponível no projeto. Base de todas as comparações de nome.

Operação e governança

Função	Comportamento
<code>onboard_clube(clube_id)</code>	Implantação de um clube novo: clona classes-modelo, cargos e documentos conforme o programa, cria o link de pré-cadastro e inicializa o checklist de onboarding. É idempotente.
<code>gerenciar_acesso_usuario(...)</code>	Concede, altera ou revoga o perfil de um usuário em um clube, validando que quem executa tem autoridade para isso.
<code>atualizar_login_membro(...)</code>	Vincula uma conta a um membro, gravando <code>membro_id</code> no contexto.
<code>aceitar_convite_responsavel(token)</code>	Materializa o vínculo de tutela e o contexto de acesso a partir de um convite válido.
<code>resetar_mfa_usuario(usuario_id)</code>	Remove os fatores TOTP de um usuário — recuperação para quem perdeu o dispositivo autenticador.
<code>registrar_auditoria(...)</code>	Insere evento em <code>auditoria_eventos</code> com ator, alvo, ação e estados antes/depois em JSONB.
<code>validar_prazo_resposta_atividade()</code>	Rejeita entrega fora do prazo, salvo se a atividade permite ou se houve reabertura explícita.
<code>idade_membro(data_nascimento)</code>	Idade em anos completos — usada nos filtros de classes agrupadas e na validação de cargo.

7.2 Triggers e cadeias de automação

Trigger	Tabela	Efeito
trg_sync_requisitos_especialidade	especialidades	Especialidade conquistada → requisitos equivalentes marcados.
trg_sync_especialidade_requisito	classes_requisitos_progresso	Requisito de especialidade concluído → conquista criada.
trg_classes_req_replicar	classes_requisitos_progresso	Replica para requisitos compartilhados entre classes.
trg_classes_req_progresso_receber	classes_requisitos_progresso	Recalcula o percentual e cria o item de investidora ao atingir 100%.
trg_sync_requisito_resposta	classes_requisitos_respostas	Resposta registrada → requisito concluído.
trg_requisito_documento_identidade	documentos	Documento de identidade entregue → requisito correspondente concluído.
trg_validar_prazo_resposta_atividade	atividades_respostas	Bloqueia entrega fora do prazo.

Proteção contra recursão

`sync_requisitos_por_especialidade` e `sync_especialidade_por_requisito` se acionam mutuamente. O corte é feito por uma verificação de **estado inalterado**: se o registro já estava com o valor final, a função retorna sem escrever, interrompendo o ciclo. É o padrão a seguir ao adicionar qualquer nova automação bidirecional neste esquema.

7.3 RPCs consumidas pelo app

RPC	Chamada em	Uso
onboard_clube	/admin/clubes	Preparar um clube recém-criado.
gerenciar_acesso_usuario	/admin/acessos	Conceder e revogar perfis.
atualizar_login_membro	/membro/[id]	Vincular conta ao membro.
aceitar_convite_responsavel	/convite/[token]	Aceite de convite de responsável.
resetar_mfa_usuario	/admin/acessos	Recuperação de segundo fator.
marcar_classe_completa	/classes/[dbvId]	Marcação em bloco de uma classe.

8 Build, Deploy e Operação

Do código-fonte ao usuário final, com os detalhes que costumam causar problema.

8.1 Ambientes

Ambiente	Web	Banco
Produção	<code>master.gsf-clubes.pages.dev</code>	Projeto Supabase de produção
Desenvolvimento	Deploy de preview (hash por publicação)	Projeto separado, populável por <code>supabase:dev:clone</code>
Local	<code>expo start --web</code>	Aponta para o projeto configurado no <code>.env</code>

8.2 Pipeline web

```
# 1 · Verificação estática - obrigatória antes de qualquer publicação
npm run typecheck          # tsc --noEmit

# 2 · Bundle estático + assets de PWA
npm run web:export          # expo export --platform web --clear
                           # + scripts/copy-pwa-assets.mjs

# 3 · Publicação
npx wrangler pages deploy dist --project-name gsf-clubes --commit-dirty=true
```

8.3 Pipeline Android

O build é feito **localmente**, sem consumir créditos de nuvem. O script `scripts/build-android-local.ps1` encapsula todo o processo e precisa ser executado **como Administrador**.

```
# Executar a partir da RAIZ DO PROJETO - nunca de dentro de C:\dev\gsfdbv
cd C:\...\fonseca-app
.\scripts\build-android-local.ps1
```

Etapa	Ação	Detalhe
1	Resolver JDK	Procura o JDK local; recorre ao JBR do Android Studio.
2	Sincronizar para caminho curto	<code>robocopy /MIR</code> para <code>C:\dev\gsfdbv</code> — contorna o limite de 260 caracteres do Windows, que quebra a compilação nativa (CMake).
3	Reinstalar dependências se preciso	Compara o hash de <code>package-lock.json</code> com o da última instalação.
4	Typecheck	Aborta o build se houver erro de tipo.
5	Prebuild	<code>expo prebuild --platform android --clean</code> regenera o projeto nativo.
6	Gradle	Gera APK e AAB de release assinados.
7	Conferir assinatura	<code>keytool -printcert</code> compara o SHA-1 com o registrado no Play Console.

Armadilha operacional documentada

A etapa 2 detecta quando origem e destino são o **mesmo diretório** e, nesse caso, pula a sincronização. Executar o script de dentro de `C:\dev\gsfdbv` faz o build usar uma cópia *desatualizada* de `app.json` — silenciosamente gerando um AAB com o `versionCode` anterior, que o Play Console então rejeita. **Execute sempre a partir da raiz do projeto.**

Assinatura de release

O `config plugin plugins/withReleaseSigning.js` reinjeta a configuração de assinatura a cada `expo prebuild --clean` — sem ele, o projeto nativo regenerado voltaria à chave de debug. A keystore tem backup fora do repositório; **perdê-la impede publicar atualizações do aplicativo**.

Checklist de publicação

1. Incrementar `versionCode` em `app.json` (o Play exige número maior a cada envio).
2. Commitar a alteração.
3. Executar o script a partir da raiz, como Administrador.
4. Conferir o `versionCode` gravado em `android/app/build.gradle`.
5. Verificar a assinatura do artefato (`apksigner verify --print-certs`).
6. Enviar o AAB ao Play Console — *upload manual*.

8.4 Migrações de banco

As 80 migrações são numeradas e **idempotentes** — usam `IF NOT EXISTS`, `ON CONFLICT DO NOTHING/UPDATE` e `DROP POLICY IF EXISTS` antes de recriar. São aplicadas manualmente pelo SQL Editor do Supabase, em ordem crescente.

Limitação de `CREATE TABLE IF NOT EXISTS`

Se a tabela já existe, o comando é **totalmente ignorado** — inclusive as *constraints* declaradas dentro dele. Foi exatamente essa a causa das especialidades duplicadas: a restrição `UNIQUE (dbv_id, nome)` constava da migração `001`, mas nunca foi criada em bancos onde a tabela já existia. Restrições devem ser adicionadas em comando próprio, protegido por bloco `DO $$ ... EXCEPTION WHEN duplicate_object THEN NULL; END $$;`.

8.5 Rotina operacional

Atividade	Frequência	Descrição
Keep-alive do Supabase	Automática	Worker da Cloudflare faz requisição periódica, impedindo a pausa do projeto no plano gratuito.
Verificação de assinatura	A cada build	Comparar o SHA-1 com <code>39:70:1A:72:A3:E0:56:5C:C4:FB:DA:40:32:A8:44:FF:4E:A4:37:0E</code> .
Backup da keystore	Contínuo	Cópia mantida fora do repositório de código.
Revisão de auditoria	Periódica	Inspeção de eventos sensíveis em <code>/admin/auditoria</code> .
Versão do termo LGPD	Sob demanda	Publicar nova versão força novo aceite de todos os usuários no próximo login.

9 Decisões Arquiteturais

Escolhas estruturais, o contexto que as motivou e o preço pago por cada uma.

#	Decisão	Motivação	Consequência
1	Base de código única para Web e Android	Equipe pequena; duplicar telas dobraria o custo de manutenção e criaria divergência funcional.	Cada função sensível precisa de dois caminhos (<code>Platform.OS</code>). Em troca, uma correção vale para as duas plataformas.
2	Backend gerenciado (Supabase) em vez de servidor próprio	Eliminar operação de infraestrutura; obter autenticação, MFA, storage e realtime prontos.	A regra de segurança precisa viver no banco (RLS), pois não há camada de aplicação intermediária controlada.
3	RLS como fronteira real de segurança	Sem servidor intermediário, o cliente fala direto com o banco — a interface não pode ser a única guarda.	171 políticas para manter. Mitigado pelo encapsulamento em 15 funções reutilizáveis.
4	Offline-first com fila de sincronização	Clubes operam em campo, frequentemente sem cobertura de rede.	Complexidade real de reconciliação de ids e conflitos, concentrada em <code>sync.ts</code> .
5	Migração multi-clubes aditiva	Havia dados de produção em uso; uma reescrita destrutiva arriscaria perdê-los.	Coexistência de <code>usuarios</code> (legado) e <code>usuario_clubes</code> . Toda checagem consulta ambos.
6	Catálogo de requisitos global, progresso por clube	Os requisitos oficiais são idênticos para todos os clubes; replicá-los multiplicaria mais de mil linhas por tenant.	Alterar o catálogo afeta todos os clubes — por isso o acesso é restrito a Admin TI.
7	Manter o agregado <code>pontos_extra</code> após criar a tabela de detalhe	Vários pontos do código já liam essa coluna (ranking, extrato offline, totais).	Redundância controlada: a aplicação mantém as duas fontes coerentes a cada escrita.
8	Build Android local em vez de EAS	Evitar consumo de créditos e dependência de fila de nuvem.	Exige ambiente Windows configurado e a etapa de sincronização para caminho curto.
9	Migrações idempotentes aplicadas manualmente	Sem chave de service-role disponível; o operador executa no SQL Editor.	Reexecução é segura, mas a ordem depende de disciplina humana.
10	Regra formativa em triggers de banco	As equivalências (especialidade ↔ requisito) precisam valer em qualquer caminho de escrita.	Cadeias de trigger exigem proteção explícita contra recursão.
11	Inativação em vez de exclusão de membros	Preservar histórico de pontuação e trilha formativa.	Consultas precisam filtrar <code>ativo</code> consistentemente.
12	União de permissões dentro do mesmo clube	Acúmulo de papéis é comum (conselheiro que também é pai).	A permissão efetiva não é a de um perfil isolado — depende de todos os contextos daquele clube.

10 Anexos

Referência rápida: migrações, rotas e vocabulário do domínio.

10.1 Índice de migrações

Faixa	Tema	Conteúdo
001–009	Fundação	Esquema inicial de clube único, seed, políticas de storage, RLS geral, documentos dinâmicos, MFA, consentimento LGPD.
010–020	Multi-clube	Programas, clubes, perfis, vínculos; permissões e modelos; pré-cadastro; fluxo de avaliação de atividades; onboarding; catálogo MDA; responsáveis múltiplos.
021–035	Refinamento operacional	Histórico de pontuação personalizada; regras de investidura; limite de anexos; itens formativos; correções de permissão; prazo de entrega; cargo adicional; funções de unidade.
036–050	Formativos e atividades	Classificação SGC; origem de especialidade; planos formativos; tema visual; timezone de prazo; reabertura; leitura e ocultação de mensagens; classe bíblica; correções para clones limpos.
051–058	Estabilização	Correções de drift de esquema; permissões de storage; itens de investidura; anexos de plano; pontuação de unidade; RPC de login de membro; configuração de faltosos.
059–070	Motor de requisitos	Estrutura de catálogo e progresso; seeds progressivos (Amigo → Líder Máster); respostas e uploads; “receber” e marcação em bloco; classes agrupadas por faixa etária.
071–086	Ajuste fino	Especialidade vinculada a requisito; <code>is_admin</code> multi-clube; marcação manual auditada; subcategorias; termo LGPD multi-clube; push; storage de fotos; autoedição de conselheiro/instrutor e de pais; extras itemizados; deduplicação de especialidades com trava única.

10.2 Índice de rotas

Rota	Finalidade
/	Painel inicial
/ranking	Classificação
/membros	Lista de membros
/membro/[id]	Ficha completa
/pontuacao	Lançamento diário
/extras	Pontos extras
/extrato/[dbv_id]	Extrato individual
/extrato-unidade/[id]	Extrato da unidade
/unidades	Gestão de unidades
/classes	Hub de classes
/classes/[dbvId]	Requisitos do membro
/classes/catalogo	Catálogo global
/classes/requisitos	Editor de requisitos
/classes/enviar	Envio de comprovação
/especialidades	Conquistas
/especialidades/catalogo	Catálogo global
/atividades	Atividades avaliativas
/calendario	Agenda
/mensagens	Mural
/relatorios	Relatórios
/importar	Importação em massa
/classe-biblica	Módulo bíblico
/perfil	Conta do usuário
/auth/login	Entrada
/auth/mfa	Segundo fator
/auth/consent	Aceite LGPD
/auth/contexto	Escolha de clube
/pre-cadastro/[token]	Formulário público
/convite/[token]	Aceite de convite
/admin/*	17 telas administrativas

10.3 Glossário

Termo	Significado
Clube	Unidade organizacional principal — o tenant do sistema.
Programa	Desbravadores (10–15 anos) ou Aventureiros (6–9 anos). Determina catálogos e faixas etárias.
Unidade	Subgrupo do clube com nome e cor próprios, liderado por um conselheiro.
Contexto	Combinação (usuário, clube, perfil) que define o que se vê e o que se pode fazer.
Classe	Etapa da trilha formativa, com requisitos próprios. Tem versão regular e avançada.
Classe agrupada	Versão condensada por faixa etária, para quem entra no clube mais tarde.
Especialidade	Certificação temática independente das classes; mais de mil no catálogo.
Requisito	Item individual a cumprir dentro de uma classe. Requisitos-raiz contam no percentual; subitens são detalhamento.
Investidura	Validação formal, pela diretoria, de uma classe ou especialidade concluída.
Plano formativo	Conjunto de atividades que, aprovadas, concluem um item formativo.
Pontos extras	Lançamento avulso com descrição livre — positivo (bonificação) ou negativo (desconto).
Pré-cadastro	Inscrição pública, sem login, sujeita à triagem antes de virar membro.
RLS	<i>Row Level Security</i> — filtragem de linhas aplicada pelo próprio Postgres.
Fila de sincronização	Tabela local que enfileira operações offline até o envio bem-sucedido.
SGC	Sistema de Gestão de Clubes — sistema oficial da denominação, referência de nomenclatura.
MDA	Manual do Desbravador — fonte oficial dos requisitos de classes e especialidades.

GSF Clubes — Documentação Técnica

Documento gerado a partir da análise do código-fonte, das 80 migrações de banco e da configuração de build.

Versão do aplicativo 1.0.10 (versionCode 15) · 23 de agosto de 2026